



Title: Classification of bird species using deep learning techniques on audio recordings of calls.

Author: Bradley Smith

Course: Data Science and Analytics

Project code: PJR40

Supervisor: Farzad Arabikhan

May 2026

Declarations

Usage	
☒	I give permission for this document to be made available to other students as examples of previous work.
Authenticity of work	
☒	I confirm that I have read the University rules in respect of plagiarism and student misconduct.
☒	I confirm that I have correctly referenced and cited any external source in any work that I produce, including the use of AI.
Ethics	
☒	I confirm that ethical approval was sought for this project regardless of whether I carried out primary research or not.
☒	I confirm that I did not carry out any primary research/the primary research carried out matches the description in my ethics application.
☒	I understand that not including the raw data can mean potentially failing the project.
☒	I understand that not obtaining ethical approval is grounds for academic misconduct and possible project failure.
Use of AI	
Artificial Intelligence (AI), such as ChatGPT, can be used as a tool to help assist and inform the initial development of your work but it is not a replacement for your own critical thinking and analysis. We require that your work is your own original content, demonstrating your knowledge, skills, and critical thinking abilities. To that end, AI-generated content must not be included in <u>any work</u> you submit for assessment unless suitably referenced. Not doing so is a breach of the University's Academic Regulations as outlined in the Student Conduct Policy and constitutes an assessment offence.	
☒	I have read the University AI policy and if relevant, I have acknowledged the use of AI in any work I produced for this project with appropriate referencing.
☒	I understand that not appropriately referencing the use of AI is an assessment offence equal to plagiarism and may result in a penalty which remains on my student record.

Name: Bradley Smith

Date: May 6th, 2026

Acknowledgements

I would like to thank Benn Jordan, an online content creator, who introduced me to the topic of bird song identification, through one of their videos on storing PNGs in birds. Numerical computations were carried out on the SCIAMA High Performance Compute (HPC) cluster which is supported by the ICG and the University of Portsmouth.

I would also like to thank the authors, development team, and contributors towards the Xeno-canto foundation, for all of their effort in preserving the millions of sounds of birds across the world.

Finally, I would like to acknowledge Andrew Gnagy, for the creation of the Nuthatch API, allowing for a more improved-looking prototype software, through the use of Unsplash, an open source community-contributed image sharing platform.

Abstract

There are numerous methods to identify bird species. Some practices involve unethically trapping birds, and causing psychological stress to them. If this is solely for the purpose of identification, concerns arise on whether this is worth the stress inflicted. This report proposes a deep learning model to identify bird species in the wild by listening to their calls, and demonstrates its predictive ability visually. The proposed model, trained on 50 species across the world, has a 2 second sound clip top-5 accuracy of over 90%. Future plans are possible for development of hybrid models to more accurately identify species.

Table of Contents

Chapter 1: Introduction.....	9
1.1. Project Background.....	9
1.2. Project Aims and Objectives.....	9
1.3. Project Constraints.....	10
1.4. Project Management and Methodology.....	10
1.5. Facilities and Resources.....	11
1.6. Legal, Ethical, Professional and Social Issues.....	12
1.7. The Report Structure.....	12
Chapter 2: Literature Review.....	14
2.1. Introduction.....	14
2.2. Research Strategy.....	14
2.3. Audio classification techniques with deep learning.....	14
2.4. Existing work for bird species classification.....	15
2.5. Issues in high class count audio classification.....	17
2.6. Summary.....	18
Chapter 3: Dataset.....	20
3.1. Introduction.....	20
3.2. Dataset Exploration.....	23
3.3. Preprocessing Techniques.....	24
3.4. Summary.....	28
Chapter 4: Model Development.....	29
4.1. Introduction.....	29
4.2. Model Design Methodology.....	29
4.3. Initial Model.....	29
4.4. Training Order.....	31
4.5. Sound Byte vs File-length Mel-Spectrograms.....	32
4.6. Pre-Existing Models.....	33
4.7. Final Model.....	33
4.8. Summary.....	35
Chapter 5: Model Application, Testing and Results.....	36
5.1. Introduction.....	36
5.2. Testing Strategy.....	36
5.3. Sound Byte Results.....	36
5.4. Sequential Aggregation and Sound File Results.....	38
5.5. Summary.....	40
Chapter 6: Prototype Software Development.....	41
6.1. Introduction.....	41
6.2. Development Environment and Framework.....	41
6.3. User Interface.....	41
6.4. Program Functionality.....	43

6.5. Code Quality.....	44
6.6. Software Testing.....	45
6.7. Summary and Further Developments.....	46
Chapter 7: Conclusion.....	47
7.1. Aim & Objectives.....	47
7.2. Risks.....	47
7.3. Project Management.....	47
7.4. Further Development/Recommendations.....	47
7.5. Conclusion.....	48
References.....	49
Appendices.....	56
Appendix A: Project Initiation Document.....	56
Appendix B : Ethics Certificate.....	65
Appendix C: Full Test Result Timeline.....	66
Appendix D: Sound Byte Predictions per Species.....	75
Appendix E: Species File Accuracy.....	77

List of Figures

Figure 1.1 - Initially proposed Gantt chart.....	11
Figure 1.2 - Revised Gantt chart.....	11
Figure 3.1 - Dataset merge procedure in file system.....	21
Figure 3.2 - Audio file count distribution per folder/species.....	24
Figure 3.3 - Sound byte counts per class before sampling was applied.....	26
Figure 3.4 - Sound byte counts per class after sampling was applied.....	27
Figure 4.1 - Iteration 1 CNN architecture.....	30
Figure 4.2 - Sample of file-length mel-spectrogram split into 2 second clips.....	33
Figure 4.3 - Final model iteration.....	34
Figure 5.1 - CNN testing over 50 classes of sound bytes.....	37
Figure 5.2 - Window sliding aggregation technique used to classify audio files (Noumida & Rajan, 2023, p. 1269).....	38
Figure 6.1 - Prototype software sample screenshot after running file successfully...	43
Figure 6.2 - Flowchart demonstrating program flow.....	44

List of Tables

Table 2.1 - Studies' dataset, class and sound file comparison.....	18
Table 3.1 - Dataset descriptions.....	20
Table 3.2 - Xeno-Canto IDs with multiple species classifications over time.....	22
Table 3.3 - Folder code species differences.....	23
Table 3.4 - Audio quality rating distribution.....	25
Table 3.5 - Audio rating classes and files after post-preprocessing.....	25
Table 3.6 - Sampling technique application results comparison.....	27
Table 5.1 - File classification and sequential aggregation function comparison.....	39
Table 6.1 - Software testing case scenarios and solutions provided.....	45

Chapter 1: Introduction

1.1. Project Background

In the modern world, many groups of people aim to identify birds for a plethora of purposes, whether to track migration patterns, to improve species conservation efforts, or to collect detailed information on a specific bird species, such as its wingspan, weight, or to check its health condition.

Typically, the term “birders” is used to describe the overarching group who perform the practices listed previously. In regards to solely identifying a bird’s species, numerous practices exist in order to identify these birds. There are many species of birds that are large enough to accurately identify with the naked eye. Examples of such birds would include crows and ravens, gulls, and eagles, to name a few. However, especially in rural, forest areas, there is an abundance of smaller birds, harder to accurately identify with the naked eye, or even with some form of binoculars/telescope. As such, one practice that is used to identify these birds is to trap the bird with a small net/cage (Liu et al., 2025). This helps secure the bird in a small area, and when a birder retrieves this net/cage, they can perform whatever research is necessary on the bird.

There is an argument to be made that this is necessary for performing detailed inspections on the bird, for conservation and health purposes. However, for the purpose of identification, there are concerns for unnecessary psychological stress being placed upon the bird. When a bird is left in a small area, and flies through a net/cage, it could go for many hours before being tended to. In (Liu et al., 2025), they mention “sustained stress behaviors (e.g. prolonged vocalizations and wing tremors)”, which, with note of the use of the word “prolonged”, strongly indicates that these behaviours are an immediate stress response, and that even initially trapping bird species can result in psychological stress responses.

Bird species can produce a wealth of calls and songs, which are used for many purposes, such as mating calls, alerting others of predators, or communicating with other members of a species. For most birds, these calls are unique to their own species. As such, the goal of this research is to introduce modern technology solutions to reduce, or even prevent, any psychological stress on these birds, by identifying bird species based on their calls alone.

1.2. Project Aims and Objectives

The final aim of this project is to develop a deep learning model to accurately classify at least 50 unique species of bird.

A suitable metric for assessing the performance of this model is a macro-averaged F1 score. The F1 score, valued in the range of 0-1 inclusive, assesses a model’s ability to classify each individual class (species), and is averaged across every class in the dataset. A high F1 score is a sign of a model which can accurately predict any

class with high accuracy. A low F1 score is a sign of a model which may predict some classes with high accuracy, but other classes with very low accuracy. As such, the aim of this research is to produce a model with an F1 score of at least 0.7.

The main objectives to reaching this step include:

- Conduct a thorough analysis on existing literature on all topics regarding deep learning techniques for audio classification, and those regarding bird species identification through audio recordings.
- Obtain a large enough training dataset for analysis.
- Perform numerous preprocessing techniques on the dataset.
- Investigate a variety of models and their performance on the dataset.
- Develop and train a final model on the dataset.
- Evaluate the model's performance through a variety of statistics, including accuracy, precision, recall, F1 score and top-5 accuracy.

A further aim of this research is to develop a prototype application which can classify an audio file recording of a bird. This application should accept an audio file as an input, and apply a trained model on this audio file, aiming to classify the bird species. The application should display the top 5 highest probabilities of bird species for the audio file.

The objectives of this further aim are:

- Investigate techniques to develop the prototype application.
- Integrate the model in a usable format into the application.
- Produce visual results in a simple-to-understand display.

1.3. Project Constraints

One constraint of this project is the application in real-world scenarios. For any given dataset, its predictive power will only apply to the species that the model is trained on. There exist over 15,000 species of bird across the planet (Barrowclough et al., 2016), and if a dataset is trained on only 20 species, there is very little real-world application value of the model. Finding a dataset that contains information on so many species would require a massively large dataset. Acquiring and working with such a dataset would be beyond the time-scope of the project.

Another constraint of the project is the computational power of any machines the model is trained on. There exist incredibly powerful GPUs capable of computing 156 trillion matrix multiplications per second (NVIDIA, 2021).

1.4. Project Management and Methodology

To facilitate the ease of managing both time and resources, the below Gantt chart was created, showing the initial forecast of the development timeline for this project.

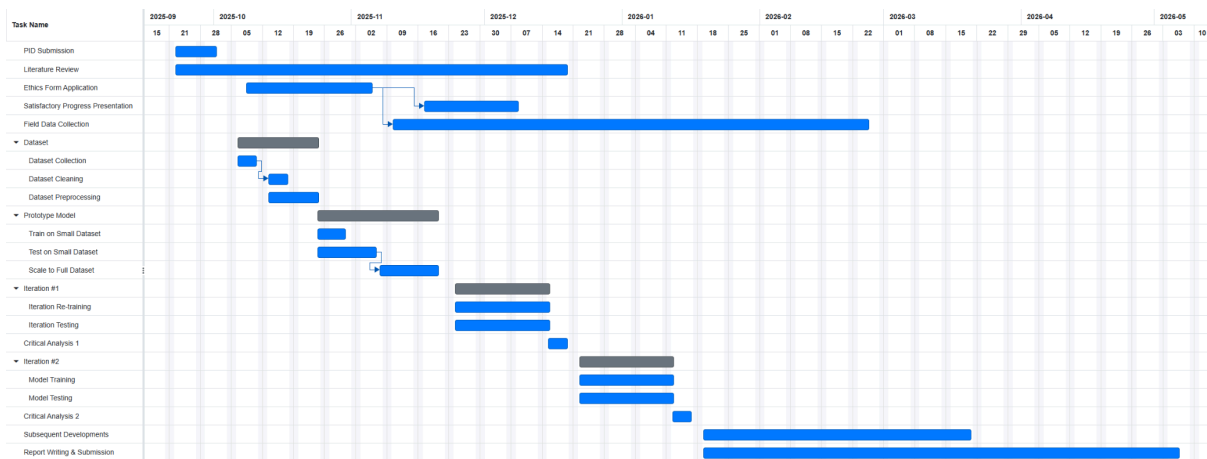


Figure 1.1 - Initially proposed Gantt chart

However, while partially into the timeline for the project, many aspects of the research were significantly incorrect. Thus, the below Gantt chart was created, illustrating a more accurate flow of events in the project’s development.

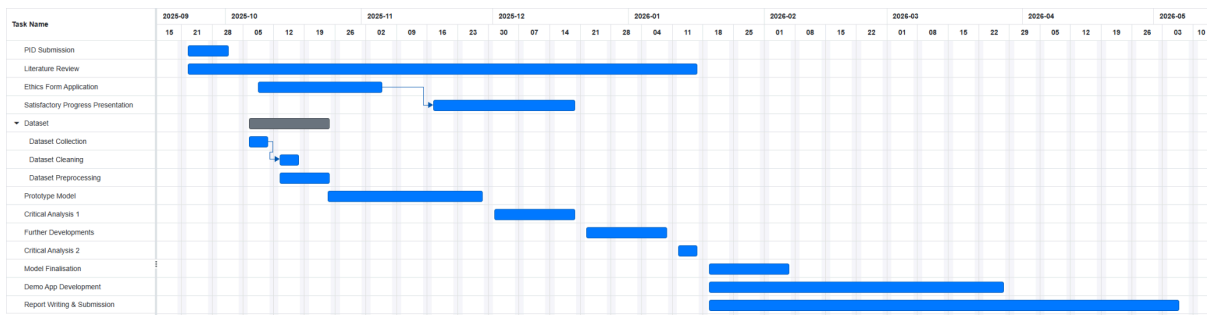


Figure 1.2 - Revised Gantt chart

This timeline was easy to follow, as it did not split large tasks into timelines of smaller tasks, which, in their own fashion, are not supposed to be handled individually, but rather in a cyclic fashion. For example, when developing and applying a model, it is trained on a training set, tested on a testing set, then refined through hyperparameter tuning to achieve more sophisticated results. This is further explored in Chapter 6. This cycle repeats until a suitable model is found, and is the cycle used in the “Model Development” sections in the revised Gantt chart.

1.5. Facilities and Resources

To minimise energy usage, this cluster was only used in circumstances where training on home devices yielded poor results after a given period of time. The closest available machine is the SCIAMA high performance compute cluster (HPC). This machine is incredibly powerful within its own right and is capable of

meeting the projects' technical needs. It utilises high-memory, high-efficiency NVIDIA A100 GPUs to perform high-speed calculations on Tensor cores, speeding the process of training machine learning models up significantly. For concerns on resource usage, it will only be used where necessary, as opposed to personal computers.

1.6. Legal, Ethical, Professional and Social Issues

The datasets used contain files only taken from the website Xeno-Canto.org. This website contains user-uploaded audio recordings of bird species from around the globe, in an aim to detect species distribution, and conserve information regarding the species.

Under the ethical guidelines of this project, the owners of the website claim “some species are under extreme pressure due to trapping or harassment. The open availability of high-quality recordings of these species can make the problems even worse. For this reason, streaming and downloading of these recordings is disabled.” in their frequently asked questions page (Xeno-canto Foundation, n.d.). This reduces, and likely negates, the chance of any bird species being under psychological stress in any recordings.

Within the dataset, some files contain metadata regarding the author of the audio (the name of the recorder), as well as the rough coordinates where it was recorded. This alone would be enough to identify a recorder. As such, to comply with General Data Protection Regulation (GDPR), should an audio file become inaccessible via a user's request to delete their information, it will be deleted on any local machines storing the datasets. Furthermore, upon the final submission and presentation of this project, all local datasets, recordings, and its corresponding metadata will be deleted, as the data used is no longer required for use.

1.7. The Report Structure

- Chapter 2: Literature Review - A critical analysis of existing literature, pertaining to all topics related to deep learning techniques for classification of bird species.
- Chapter 3: Dataset - A detailed review of the datasets used in this study, and similar datasets used in studies similar to this one.
- Chapter 4: Model Development - A detailed description and thorough analysis of the datasets used in the project. Plus an in-depth comparison of the models used throughout the project, including each model's architecture, and the results obtained through using each model on the dataset.
- Chapter 5: Model Application, Testing & Results - The final model used for the project for classification of bird species, with details of any changes made to existing iterations, with its results presented in a visual format, alongside its architecture.
- Chapter 6: Prototype Software Development - A timeline of the research conducted and techniques used when developing the prototype application to

showcase the model's capabilities. It also covers use case scenarios, and software error handling.

- Chapter 8: Conclusion - A critical review of every method used throughout this project, was the initial problem solved, which issues could have used improvements, what benefits this project has brought to any prospective groups as well as any future recommendations for research into this area.

Chapter 2: Literature Review

2.1. Introduction

This chapter discusses the strategy used to find existing literature relevant to the topic of this research, which led to various papers covering the same topics, regarding deep learning techniques used to classify sound files, covering numerous areas of research into natural environmental sound identification, wherein some papers looked specifically into bird sound identification, on similar data sets to the ones used in this research. Due to the nature of this research, papers that delved into high class audio classification were also investigated, as there can be some techniques used to ensure a higher accuracy classification.

2.2. Research Strategy

The general research strategy involved finding papers which covered common tools and techniques for audio classification. The EBSCO Discovery service provided by the University was used, wherein keywords such as “deep learning”/“machine learning” combined with “audio”/“sound” and “birds”/“bird species” were used to filter results. Due to the nature of technological developments, only research papers that had been published and peer reviewed within the last 5 years were initially used. From these research papers, they often made mention of other papers where similar techniques with varying results were used. This served to establish a baseline for the tools and techniques that could be used in this research project.

While extensive filtering into specific papers covering the topic of this research was an option, the previously mentioned strategy led to a plethora of papers that covered similar technologies that could be used, and thus was the only strategy used to find existing literature.

2.3. Audio classification techniques with deep learning

One paper that was incredibly useful, serving as a comparison of strategies for sound classification investigates environmental sound recognition on embedded devices. (Gairí et al., 2025)

This paper investigates 34 applications of environmental sound classification, such as traffic management (Montino & Pau, 2019), speaker recognition for biometric authentication (Laksono & Prasetyo, 2023), and most notably bird song recognition (L. Duan et al., 2024; Huang et al., 2024). All of these applications were performed on “edge” devices, being lightweight devices with very low computational complexity, such as smartphones on the upper-end of power (Choudhary et al., 2022; Hyun Choi et al., 2024), and Arduino Nano 33's, (Lin et al., 2024; Maayah et al., 2023) with only 256kB of random access memory (RAM), and a CPU capable of performing only 64 million instructions per second (MIPS) (Arduino, n.d.). While edge devices do not have the computational complexity of a modern CPU, the models themselves were all trained on complex machines, and compressed through numerous optimisation and compression techniques, with some common tools being 8-bit quantisation

(Jacob et al., 2017), and pruning, where individual node connections that provide little change to model outputs are removed. Due to the nature of the task in this research, it should not be necessary to compress the model, as this resulted in a decrease in accuracy and other metrics across most of the studies investigated in the paper. Were this a real-world application where this was to be field tested, compression would be appropriate, as smaller devices would not have enough memory to store the weights and transform any input data properly. However, some devices, such as Raspberry Pis, have a large memory size, up to 4GB seen in (Wyatt et al., 2021), which allows for larger scale models to not require any form of compression for real-world application.

While the studies are all performed on audio files and sound clips, the models used interpret the data in a variety of formats. Few studies analyse raw audio data without any transformation (Choudhary et al., 2022; Ko et al., 2022), which was particularly useful in the high-class study performed by (Choudhary et al., 2022). The most commonly used model input was transforming audio data into mel-spectrograms, where a logarithmic transformation is applied to frequencies into filter banks, and a spectrogram is formed from these filter banks. These were analysed through convolutional neural networks (CNNs), because a key design principle for this network type handles spatial data through convolutional filters (kernels). This was used in 19 of the analysed studies, in a variety of input shapes. This would serve as an indicator to be a starting point for model development. All other techniques were used in only singular studies where there was little to no applicability into bird species identification.

Within these studies, numerous preprocessing techniques were applied and analysed. Some common and notable techniques used include decibel range clipping (Wyatt et al., 2021), as well as amplitude threshold filters (Ko et al., 2022), both used for handling “silent” or quiet data, where there is no classifiable noise. Other cropping techniques were used on audio files and their transformations, including trimming and/or padding of sound files, for uniform length sound clips (Laksono & Prasetyo, 2023), as well as cropping spectrograms and mel-spectrograms for filtering higher frequency noises (Hyun Choi et al., 2024; Wang et al., 2024).

2.4. Existing work for bird species classification

There exists numerous pieces of literature covering bird species classification. One such paper investigates multi-label classification of soundscape files. R. Rajan and A. Noumida, alongside other authors, have conducted numerous studies into the classification of bird species with deep learning methods.

Their first work (Noumida & Rajan, 2021), investigates isolated recordings of bird calls, wherein a singular audio clip contains one bird’s call, with little to no surrounding noise from other environmental sources or bird species. They propose a solution for classification through the use of convolutional neural networks (CNNs) and deep neural networks (DNNs). They also experiment with transfer learning,

which utilises commonplace existing CNNs with predefined weights from other datasets, which includes ResNet50 (He et al., 2015), Inception-ResNetV2 (Szegedy et al., 2016), and VGG-16 (Simonyan & Zisserman, 2015), in order to compare a proposed model on a given dataset. They propose a unique CNN architecture to handle the classification of this audio file, shown in Table 1 of the study (Noumida & Rajan, 2021, p. 253). In total, they investigate 644 unique sound files across 10 bird species. For all of the CNN architecture-based models used, they are all trained on mel-spectrogram transformations of the audio files. Applying 5 unique models to the dataset, it is seen that ResNet50 performed the best, with an accuracy of 96.3%, followed by their proposed CNN, VGG-16, and their proposed DNN. From this, it can be concluded that for audio signal classification, convolutional neural networks on mel-spectrograms lead to the highest accuracy over other neural network architectures.

The previous paper compares their results of the ResNet50 model to that of the same ResNet50 model trained by (Mangalam Sankupellay & Konovalov, 2018). In Sankupellay and Konovalov's research, they perform classification of bird sound files across 46 different bird species, after transforming the files into simple spectrograms. Their results showed ResNet50 performed with an accuracy of 65%. While this shows that numerous preprocessing techniques - including the transformation of clips into Mel-spectrograms - can significantly improve the accuracy of a deep convolutional model, there are numerous differences in the two studies worth highlighting. In Sankupellay and Konovalov's study, the number of species classification is performed on is 46, whereas Rajan and Noumida's study investigates only 10 bird species. Mathematically, an extra 36 classes to predict increases the error rate of the exact same model's predictive ability significantly. Alongside this, while it was not mentioned in Rajan and Noumida's 2021 study, a later investigation on multi-label classification, by the same authors, (Noumida & Rajan, 2023), identified that the qualities of the same 10 species tested in their 2021 study all represented a broad range of bird call patterns. This can be seen in Table 6 of (Noumida & Rajan, 2023, p. 1266). The visual differences in each of these call patterns could attribute the high accuracy of their 10-class ResNet50 model, as the CNN would be able to appropriately differentiate between the numerous types of visual differences in the mel-spectrogram images, examples of which were identified in (S. Duan et al., 2011, p. 516).

Noumida and Rajan's 2023 research on multi-label classification mentioned previously is a critical paper which experiments with deep convolutional neural networks (DCNNs). They also apply recurrent neural network architecture in the form of recurrent neural networks (RNNs) and long short-term memory (LSTM) on mel-frequency cepstral coefficients (MFCCs) (REFERENCE THIS) from the audio signals. In addition to the other transfer learning models experimented in previous studies, InceptionV3 (CNN) (Szegedy et al., 2015), EfficientNet (CNN) (Tan & Le, 2020), and Vanilla-DNN (DNN) are also experimented with. A key feature of this study is the inclusion of sequential aggregation for producing a multi-label classification. They split audio files into windows, transforming and inputting into the models for a given output, which is then aggregated by summing outputs and normalised, to get a multi-label classification for the entire audio file. While this

technique is useful for multi-label classification, this technique can also be applied to single-label data, as the process does not change how many output classes are required per file. While the study applied time and frequency masking to their mel-spectrograms as data augmentation techniques to create extra training data for the CNN-based models, it is unknown whether or not this helped with the training of their models. No metrics are mentioned regarding the training accuracy of the CNN-architecture models, yet the LSTM models trained on MFCCs reach a training accuracy of 100%. This raises the question of how appropriate data augmentation is on a specific model's data, in which overfitting is shown in other models. This even further questions the validity of including the CNN-based models, when the amount of training data is so small.

2.5. Issues in high class count audio classification

Studies conducted into the feasibility of high class audio classification usually result in models that are not very accurate. As seen in (Noumida & Rajan, 2021), they are able to achieve a classification accuracy of 96.3% with the ResNet-50 model on 10 classes. In (L. Duan et al., 2024), they perform an analysis of multiple neural networks on a dataset of 20 classes of birds. Their proposed "SIAlex-5" model performed with an accuracy of 93.66%, and a ResNet-50 model performed with an accuracy of 93.43%, with a ResNet-34 model performing with an accuracy of 92.86%. While their model was not performed on the same dataset, as their data source was different to that of Rajan & Noumida's, found on the Xeno-Canto website¹, this is an indicator that models trend downwards in accuracy as the number of output classes increases, for similar models.

To look at an example of a study conducted on a significantly larger amount of classes, (Choudhary et al., 2022) investigated the FSD50K audio clip dataset (Fonseca et al., 2020), containing 51,197 sound clips of length 0.3-30 seconds, over a total of 200 unbalanced classes. This data is multi-label, with an average of 1.22 labels per sound clip. Their best-performing proposed model achieves a mean average precision (mAP) of 0.4677. This implies that the model correctly predicts the correct class 46.77% of the time for all predictions of that class on average. While this metric is not a direct representation of accuracy, it can be inferred that it does not perform nearly as well as the SIAlex-5 and ResNet-50 models on 20 and 10 classes respectively. This is further evidence to suggest that increasing the number of output classes trends downwards in the model's predictive power.

Finally, a study performed by (Laksono & Prasetyo, 2023) performed an analysis of a proposed CNN model for speaker recognition over two datasets consisting of 30 classes and 125 classes. While this study did not offer any baseline comparison of their proposed model to existing popular models such as ResNet-50 or VGG-16, their proposed model achieved accuracies of 77.62% and 56.40% on the 30 class dataset and 125 class dataset respectively.

¹ <https://xeno-canto.org/> | xeno-canto

While the studies by (Choudhary et al., 2022; Laksono & Prasetyo, 2023) are not performed on bird sound datasets, they consist of qualities that uniquely identify specific sound features that would distinguish one class from another. (S. Duan et al., 2011, p. 516)

2.6. Summary

Table 2.1 shows a summary of the scope of the datasets used, in comparison to the final dataset used in this study. It is evident that the scope of the dataset used in this research will be of higher complexity for classification compared to other datasets, and extensive experimentation of multiple techniques in the model development and application will be necessary to maximise the applicability of any models used.

Research	Dataset	Total Classes	Total Sound Files
This Study (Final Model Iteration)	Xeno-Canto ²	50	2362
(Noumida & Rajan, 2021)	Xeno-Canto ²	10	644
(Mangalam Sankupellay & Konovalov, 2018)	Xeno-Canto ²	46	2814 ³
(Kumari et al., 2019)	ESC-50 (Piczak, 2015)	50	2000
(Mou & Milanova, 2024)			
(Choudhary et al., 2022)	FSD50K (Fonseca et al., 2020)	200	>51,000
(Laksono & Prasetyo, 2023)	VoxCeleb (Nagrani et al., 2017)	125	15,007

Table 2.1 - Studies' dataset, class and sound file comparison

While there were a wide variety of studies conducted on classification of bird species with deep learning techniques on audio recordings, the number of classes analysed in the respective studies in comparison to the 992 unique bird species in the dataset used in this research is only a tiny fraction. The largest problem with this is there is a

² These used a combination of sound clips from previous BirdCLEF/LifeCLEF challenges authorised by the Cornell Lab of Ornithology, but all sound clips are taken from the Xeno-Canto website.

³ 2814 *spectrograms* were used, but it is not mentioned how many sound files were used. Thus, 1 spectrogram per sound file is assumed to have been used.

distinctive lack of real-world applicability. There are over 10,000 unique species of bird in the world, and as can be seen through the 200-class study by (Choudhary et al., 2022), in which a mAP of 46.77% is attained. This is not specific to bird classification, but serves as a guideline towards what is to be expected through research into high-class bird species classification.

Chapter 3: Dataset

3.1. Introduction

There are a total of 5 datasets used in this study. Each dataset is sourced from the Cornell Lab of Ornithology's Kaggle competitions site (Cornell Lab of Ornithology, n.d.). The source of all of the data comes from Xeno-Canto, mentioned in previous chapters. Each file is located under a sub-folder corresponding to the shorthand code for a bird species, with each file being of the format XC[number].[ogg/mp3]. The "XC[number]" corresponds to the Xeno-Canto audio file identifier.

Table 3.1 below summarises the properties of each of the 5 datasets used in this study. The final row is the resulting dataset after combining the 5 datasets with the procedure highlighted in Figure 3.1.

Name	Year	Dataset Size (GB)	Unique Classes	Total Files	File Type	Notes
Birdcall Identification	2020	23.6	264	21,375	mp3	
BirdCLEF 2021	2021	39.2	397	62,874	ogg	
BirdCLEF 2022	2022	6.14	152	14,852	ogg	
BirdCLEF 2023	2023	4.92	264	16,941	ogg	
BirdCLEF 2024	2024	21.8	182	24,459	ogg	2/3 of the dataset contains unnecessary data.
This Study	2026	57.4	992	112,861	mp3, ogg	

Table 3.1 - Dataset descriptions

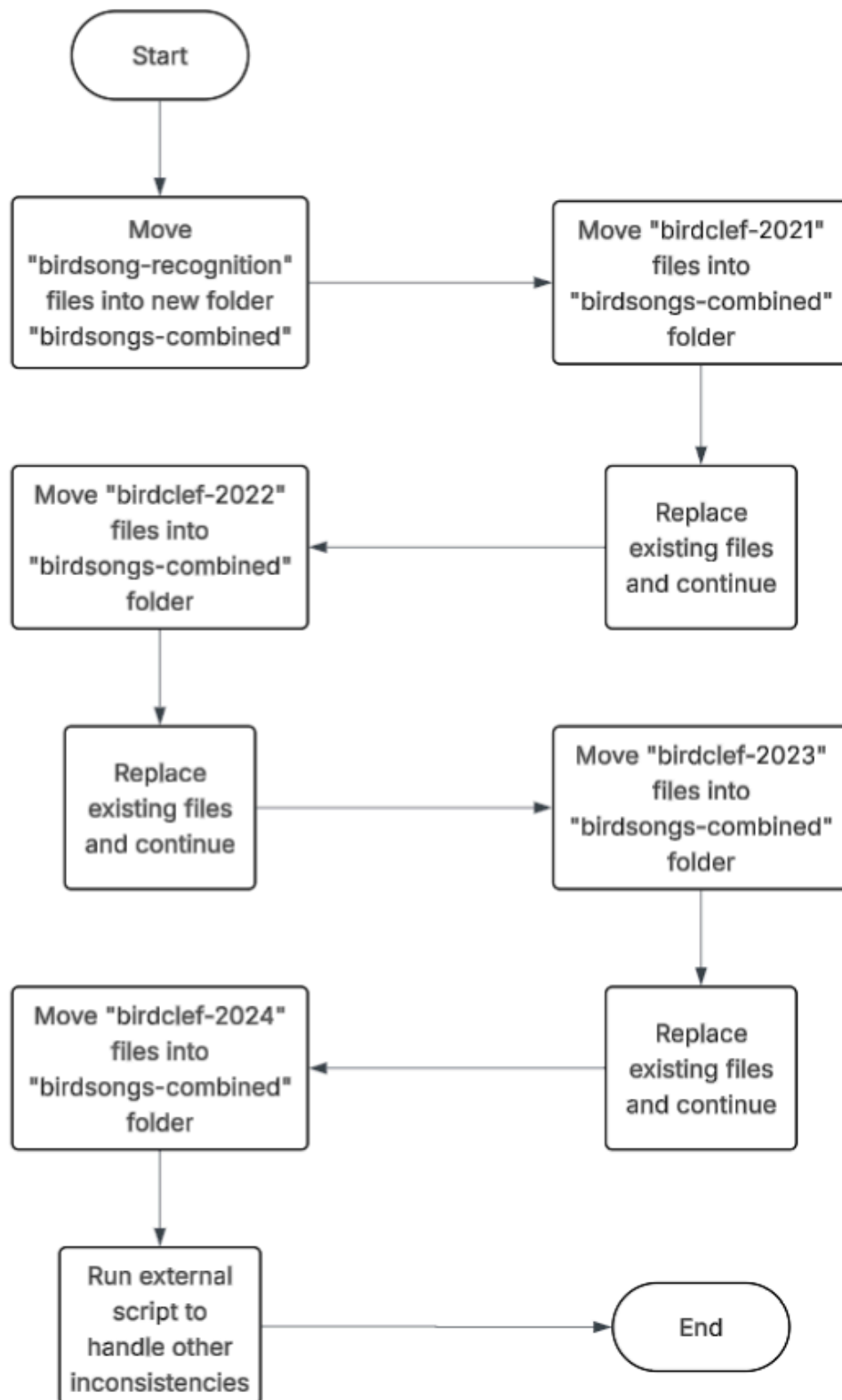


Figure 3.1 - Dataset merge procedure in file system

The biggest risk when merging these datasets was handling any duplicate XC[number] files with multiple file types. Multiple techniques were considered, such as excluding the 2020 dataset entirely, or converting the files to ogg before merging them. Ultimately, a simple script to check for existing ogg files for any IDs with both an mp3 and ogg file after merging. If a XC ID contained both an mp3 and ogg file, the mp3 file was deleted. This, combined with handling specific edge cases where 6 specific IDs had multiple classifications (due to species identification changes handled over time, covered in Table 3.2), resulted in a final dataset of 112,861 unique ID files, over 992 unique classes.

XC ID	Old Species	New Species
406586	Eastern Meadowlark	Western Meadowlark
136316	Louisiana Meadowlark	Rufous-capped Warbler
502062	Louisiana Meadowlark	Northern Waterthrush
376774	Prairie Warbler	Field Sparrow
475655	Red-breasted Merganser	Northern Shoveler
351819	Western Kingbird	Cassin's Kingbird

Table 3.2 - Xeno-Canto IDs with multiple species classifications over time

This could raise validity concerns in the future, as some species which may have been identified as one species in the current time of the research, may have changed, and negatively influenced the model. Though, given the size of the data, this is unlikely to be the case, unless it was on a much larger scale than one or two sound files over 100,000 total.

Another concern faced when merging datasets was that 1,014 unique species names were present, despite only 992 folders existing in total. Investigation of these 22 extra species names showed that multiple folders had a name change over time. 17 of these 22 had local name changes for a species, or a slight spelling/hyphenation difference (grey vs gray, Red breasted vs Red-breasted, etc.) and were handled accordingly. Hyphenations were taken over non-hyphenated forms, and spellings took the British English spelling (grey, instead of gray). 5 remaining species had entirely different classifications, which are covered in **(Clements et al., 2025)**, and Table 3.3 summarises the changes. Species highlighted in bold text were chosen as the singular species to cover the folder code.

Folder code	Species 1	Species 2	Notes
-------------	-----------	-----------	-------

amepip	American Pipit	Buff-bellied Pipit	The buff-bellied pipit was the former name of the American Pipit.
banswa	Bank Swallow	Sand Martin	The sand martin is known as the bank swallow in the Americas. Bank Swallow chosen due to folder code being a short-form already.
gnwtea	Eurasian Teal	Green-winged Teal	Same reasoning as above.
yelwar	Mangrove Warbler	Yellow Warbler	Mangrove warbler is a subspecies of the yellow warbler.
yerwar	Myrtle Warbler	Yellow-rumped Warbler	Myrtle warbler is a subspecies of the yellow-rumped warbler.

Table 3.3 - Folder code species differences

3.2. Dataset Exploration

Across the entire dataset, a total of 1,570 hours, 16 minutes and 44.76 seconds of audio was present, before preprocessing. The distribution of files per folder is shown in Figure 3.2 below, where a large number of species only have less than 200 sound files. The total sound length per species was not calculated, however, it could be assumed that it follows a roughly similar distribution. This negative skew could lead models to over-classify those species with higher counts of sound files.

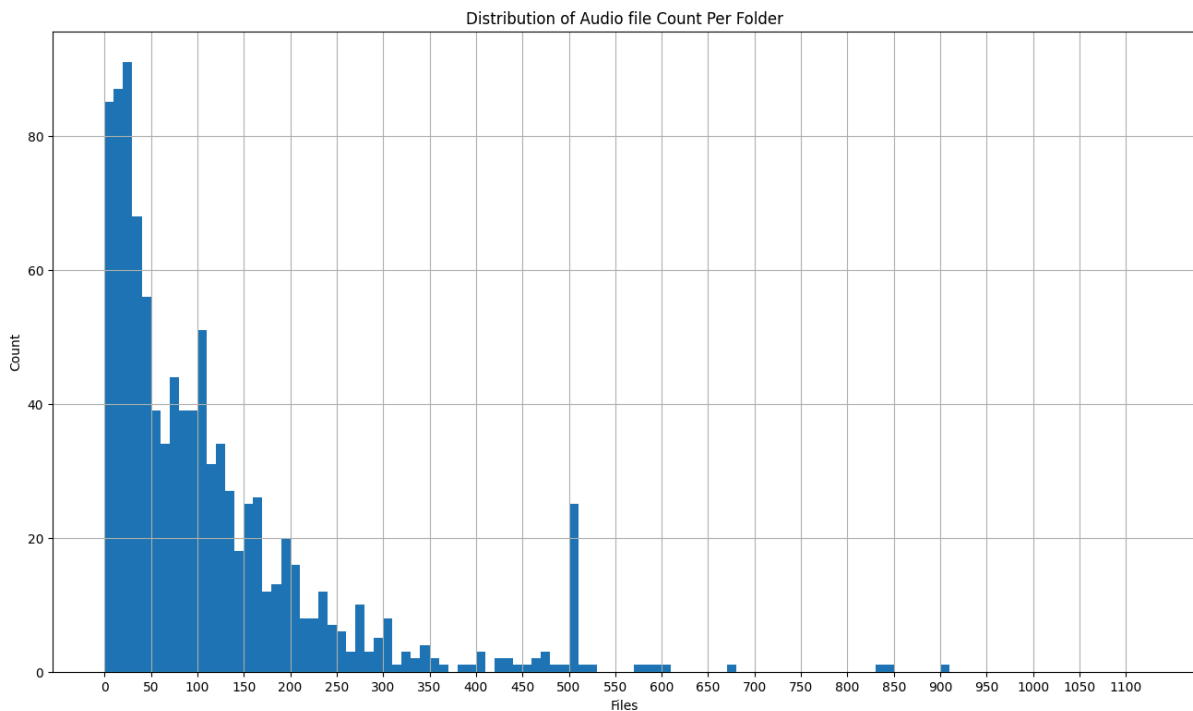


Figure 3.2 - Audio file count distribution per folder/species

3.3. Preprocessing Techniques

Within the dataset, there were several attributes of the data that needed to be filtered from the initial 112,861 audio files. This includes:

- Handling of multi-label classification files.
- Filtering of high-quality from low-quality sound files.
- Removal of classes with little to no data available.

The techniques listed above were performed in the given order. After the five datasets had been combined to include unique audio files, the total number of audio files was 112,861. Of this, 27,195 were listed as having multiple labels, wherein instances of multiple species of bird were detected in the audio recording. While each file has a primary label (main bird species) and secondary labels (subsequent bird species), it is safer to exclude these files as the classification task is single-label classification. Having extra bird species within an audio file could mislead any model weights towards predicting a specific class. Following the removal of these multi-label files, the dataset now consisted of 85,666 single-label audio files.

Each of the audio files contained a “rating” letter, as is defined on the Xeno-Canto website, where E (0.0) is the lowest quality, being “barely audible”, and A (5.0) being “loud and clear.” (Xeno-canto Foundation, n.d.) This serves as a guideline on which audio files to use for the most accurate classification possible. The website mentions rating B (4.0) may contain “some interference with other sound sources.” This, combined with the distribution of ratings in Table 3.4, can be used to determine a baseline minimum rating of 4.0 for all sound files used in the dataset. This takes the total number of audio files in the dataset to a maximum of 59,327.

Rating	Audio File Count	Cumulative Count	Total Files >= Rating
0.0	3,907	3,907	85,666
0.5	88	3,995	81,759
1.0	664	4,659	81,671
1.5	419	5,078	81,007
2.0	2,756	7,834	80,588
2.5	2,066	9,900	77,832
3.0	11,391	21,291	75,766
3.5	5,048	26,339	64,375
4.0	28,782	55,121	59,327

4.5	5,028	60,149	30,545
5.0	25,517	85,666	25,517

Table 3.4 - Audio quality rating distribution

The final dataset-specific preprocessing technique applied was an investigation of file count distribution per class. For ratings 4.0, 4.5, and 5.0, Table 3.5 summarises the file count distribution per folder after all of the aforementioned preprocessing techniques were applied.

Rating	Total Files ≥ Rating	Total Classes	≥10 Files per Class	Total Files After Filtering ≥ 10 Files per Class
4.0	59,327	976	833	58,589
4.5	30,545	951	659	29,239
5.0	25,517	945	638	24,163

Table 3.5 - Audio rating classes and files after post-preprocessing

Experimentation with models on each dataset will determine the use of models going forward. The number of files per class is sufficiently large enough in the 5.0 rating dataset, with 25,517 files across 945 classes, for it to be automatically used for analysis. However, with the 4.0 and above rating dataset containing over double the number of files across only 3% extra classes could be useful in training the model, as neural networks, particularly CNNs, benefit from a larger amount of training data.

Another preprocessing technique used in file handling was sampling. Class imbalance is an important factor to consider in neural network training, as high frequencies of a specific training class can skew a model to predicting that one class, as loss calculation would be minimal. There are numerous ways to tackle class imbalance, through undersampling and/or oversampling. Random oversampling and random undersampling were experimented with, as existing algorithms such as SMOTE (Chawla et al., 2002), cannot be accurately implemented on audio data, due to its nature in being field sample recordings, from a plethora of environments. These random/naive algorithms simply duplicate/delete existing data to oversample/undersample respectively. Despite being a simple algorithm, this can lead to detrimental effects in model predictive ability, as it cannot account for minor differences in shapes of bird calls.

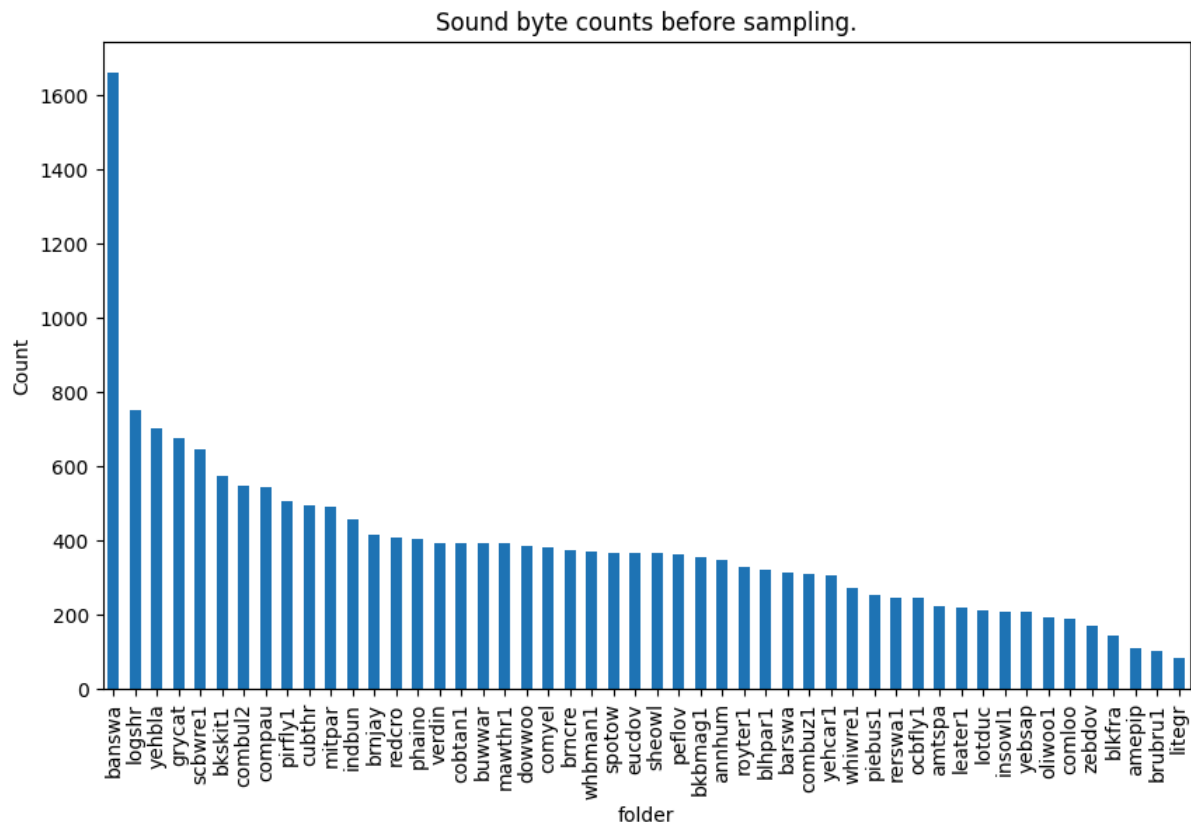


Figure 3.3 - Sound byte counts per class before sampling was applied.

Due to the limited number of samples across audio files, applying random undersampling to match the count of the minority class in a sample of 50 classes drastically reduced the training data, and as such, resulted in much lower accuracy and predictive capability for the model. In similar fashion, applying random oversampling to match the majority class drastically increases duplicate amounts of smaller classes, and the volume of data results in training taking a significant amount of time. This was identified by (Chawla, 2005), where it was highlighted that “The random undersampling method can potentially remove certain important examples, and random oversampling can lead to overfitting.”

Thus, a hybrid approach was constructed, which undersamples the majority classes and oversamples the minority classes to match the median class count across all classes. This technique will be referred to as “median” sampling. It could be argued that this technique is not beneficial for the dataset, as it applies both strategies which may lead to biased results towards one class, with unnecessary duplicates also existing in the data set.

Table 3.6 summarises an application of random oversampling, random undersampling, and “median” sampling across 25 classes.

Sampling Technique	Test Size	Total Sound Clips	Peak Test Accuracy	Train Accuracy
Oversampling	30%	70,325	38%	47%
Undersampling	30%	17,666	36%	94%
Median sampling	20%	12,300	66%	100%

Table 3.6 - Sampling technique application results comparison

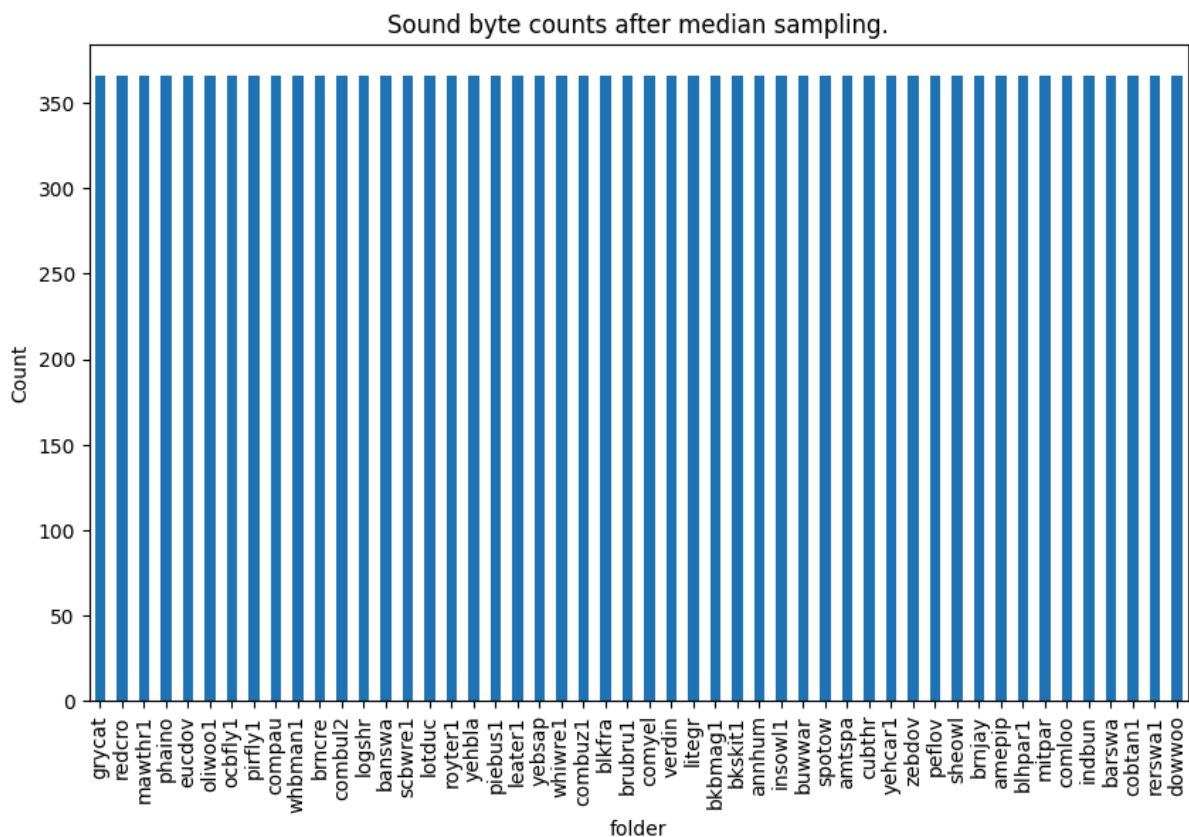


Figure 3.4 - Sound byte counts per class after sampling was applied.

It can be clearly seen that despite reducing the total number of sound clips by a significant margin in comparison to undersampling, and especially oversampling, the testing and training accuracy of the same model had increased by a large amount. The training accuracy being 100% is evident of overfitting being present, however, due to the smaller training size, it was able to fit the entirety of the training data set. The higher accuracy of this model could also be attributed to the reduced test size. Though it could be argued that, given the volume of the data set, the relative amount of training data may have been reduced by 33%, but the absolute amount of training data is still high enough to properly train the model fully.

Finally, the last issue encountered when handling sound bytes of audio files was to determine if any specific sound file was “silent.” Having silent (or low ambience, with no actual bird calls) audio can negatively skew a model’s predictive ability, due to no actual presence of sound data being classed as a specific bird species. Thus, a naive approach was developed to handle this. Since very little audio data was truly silent with very low/zero amplitude, two metrics were used to identify silent audio. A mean decibel level was calculated from the sound byte, in decibels, and a variance in pitch over the entire sound byte. Sound bytes that fell below this mean decibel threshold and also had a low enough variance (meaning consistently quiet sound) were classified as “silent” audio.

3.4. Summary

This dataset offers a wealth of data for training the model. It is diverse, containing a large range of different species types which should allow the model to accurately identify different species apart. Further development could have been applied with exploration of the data, however an understanding of the files can only go so far beyond class distribution.

Chapter 4: Model Development

4.1. Introduction

This chapter explores the various technologies that were implemented for the training and testing of the model, and how the experimentation of hyperparameter tuning affected model accuracy.

4.2. Model Design Methodology

The model design methodology chosen for this research follows that of CRISP-DM. The CRISP-DM (Cross Industry Standard Process for Data Mining) (Shearer, 2000). The methodology outlines the steps of the cycle for data mining and model development. Business understanding and Data understanding are a back-and-forth section which covers all the needs and description of system requirements, as well as a detailed breakdown of all the data involved. In this research project, the “business understanding” is the final aim of this project, to develop a suitable model. After the data understanding phase, a data preparation and modelling back-and-forth stage is entered, where data is transformed and fed into a suitable model architecture. Once a suitable model has been formed from existing data, the model is evaluated (in this case, against species-specific audio files). If the evaluation result is not suitable, then development returns to the business understanding phase, otherwise the model is deployed. This process repeats for any model development.

This was suitable enough for this research project, as results generated from any changes in the model would lead to research into additional tools and techniques to apply to the model.

4.3. Initial Model

For a study of feasibility, a simple convolutional neural network was constructed, with its architecture shown in **Figure 4.1**. This will be referred to as “Iteration 1”. This model was used purely for experimentation with hyperparameter tuning, and testing the physical capabilities of model training. This also served as a method for testing if specific preprocessing techniques had an effect on the output of the models. Numerous metrics were assessed in the training of the model. The important metrics were the time taken to train the model, the best model’s testing accuracy, with its corresponding training accuracy, as well as the top 5 accuracy of the model.

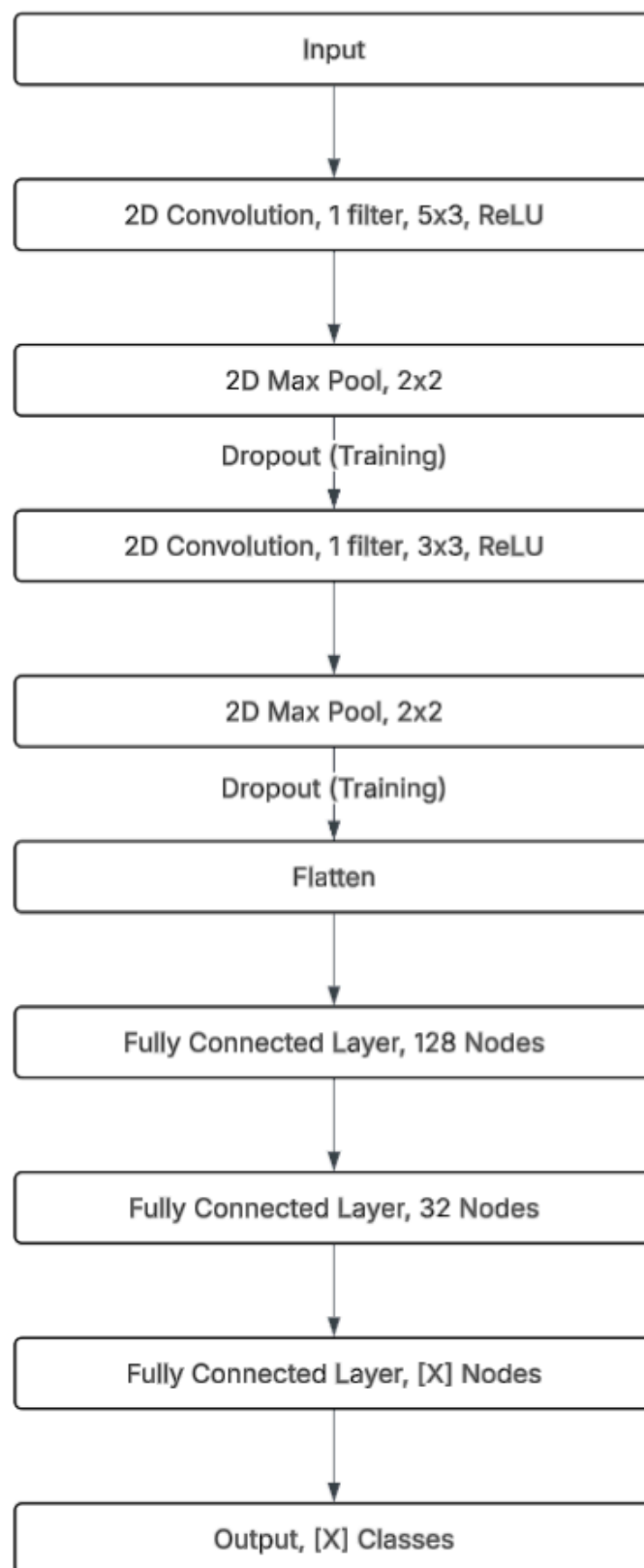


Figure 4.1 - Iteration 1 CNN architecture

The top 5 accuracy of a model is the proportion of which a target label appears in the top 5 predictions of a model's output. This is more commonly seen in top 5 error, which assesses how often it does *not* appear in the top 5 predictions of a model. This is an extended form of accuracy, and is especially useful in high-class predictions, as it serves as a metric indicating the predictive capability of a model past the highest valued label, indicating that learning capability exists within the model but is not yet fully tuned.

This is useful as a metric, because for bird species in particular, species classification can be geographical. Two bird species can have an identical sounding call, but be located in different geographic locations. For example, a hooded crow and a carrion crow (RSPB, n.d.). This can affect the predictive capability of a model, as this analysis only investigates bird sounds independently from geographic locations of calls. In a real-world application, geographic location can be inferred from the location of a device making a recording. This would help direct a model towards making correct predictions. Thus, it would be safe to conclude a further investigation towards identifying multiple bird species with identical calls in different geographic locations is strongly recommended.

4.4. Training Order

One technique that was applied when training all models was to test it on a specific order of species when loading all the sound data.

As the dataset contains a minimum of 638 classes in its entirety (when viewing files with a rating of 4.0 and above), with a maximum of 992 folders, there are a variety of species with similar, if not identical, calls. This will skew the accuracy of a model in the negative direction, as identical audio patterns across multiple classes would only make a model less accurate, since the probability of predicting any class would be a uniform chance. Due to the scale of the dataset, comparing audio calls by hand would be a very time consuming task, so in an attempt to stratify training, bird genus' were investigated.

All of the 992 species in the dataset follow a scientific binomial nomenclature. For example, the common name "carrion crow" has the scientific name "corvus corone". This means the first word (the generic name) identifies the genus of a bird. The second word identifies the species with that genus. There were a total of 531 unique genera within the dataset. When generating the order of the dataset, all 531 genera had a singular species before any duplicate genera were inserted. After this, any genus with at least 2 occurrences had another species inserted, and then all genera with at least 3 occurrences had another species inserted. This process repeated until all 992 species were in order.

This technique meant that results of training a model would not be due to the properties of classes analysed, but rather the intricacies identified and trained through models. This however, does not negate any risks associated with

classification and training, only mitigates. If all the files of a folder are to be analysed, this is even less of an issue in determining model accuracy differences, however if, for space conservation, up to a certain number of files per folder are analysed, this can introduce unnecessary randomness into training the models.

4.5. Sound Byte vs File-length Mel-Spectrograms

There was conflict between whether to use sound byte mel-spectrograms, or ones generated on the entire file, and then split up into equal length segments. With identical sound byte durations, the size of the input data would be equal regardless, but the biggest concern was the relative volume (the colour on the image, as seen in Figure 4.2). If a file was to be split into sound bytes, and then have a mel-spectrogram transformation applied, all of the decibel levels would be relative to the maximum level in that sound clip alone. However, if the entire file were to be transformed first, each level would be relative to the maximum frequency across the entire file.

It was decided that file-length mel-spectrograms would be applied, as this would be able to identify silence/ambient noise much more accurately. If, instead of using file-length mel-spectrograms that were split up, files were split up before applying the transformation, the darkness in Figure 4.2 would appear much brighter, as this would be the ambient noise with more prominence.

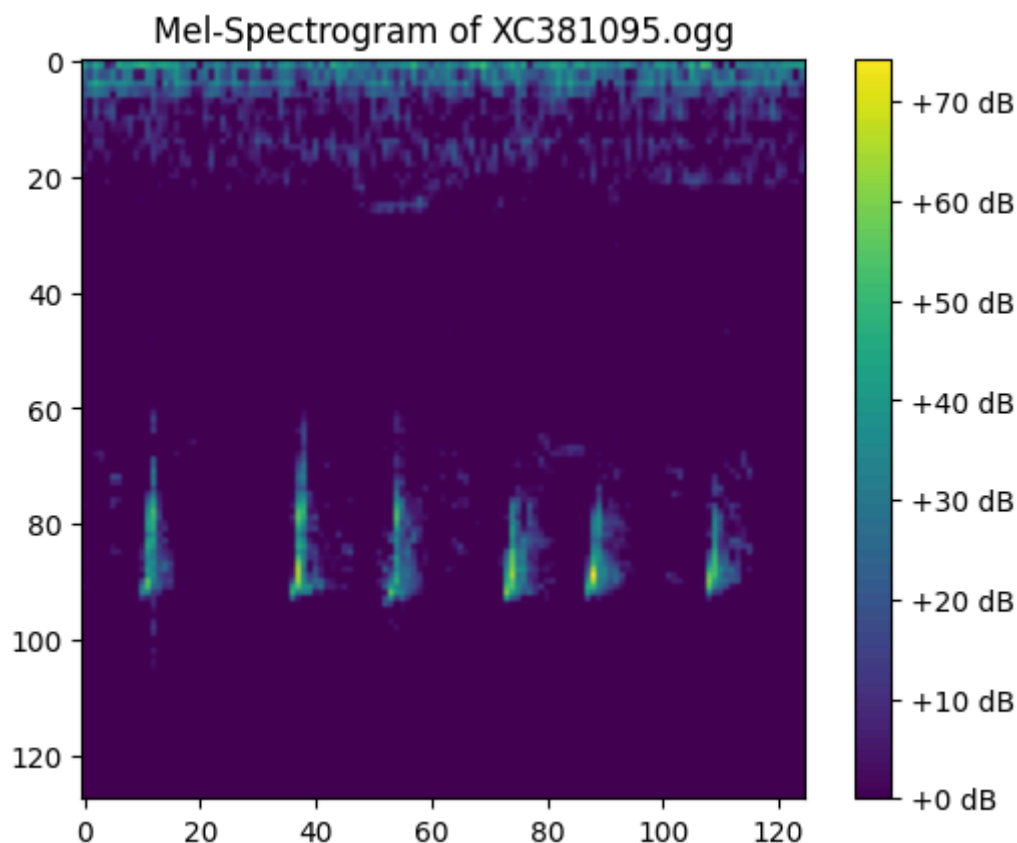


Figure 4.2 - Sample of file-length mel-spectrogram split into 2 second clips.

4.6. Pre-Existing Models

There were many pre-existing models considered for application. A variety were highlighted in Chapter 2, including VGG-16, ResNet models, and EfficientNet models. Among these, VGG-16 has the least layers, and is the least complex of the listed architectures. To fit with the shape of training data, the pre-existing model layers were configured to accept a variable input size, instead of the suggested input shape of 224x224x3 (Simonyan & Zisserman, 2015). As a result, this could be tested side-by-side with other models.

However, upon implementation and testing, accuracy and loss always fell to a local minimum, resulting in testing and training accuracies of approximately 2-3%. On top of this, due to the number of layers and weights, local memory storage was very high, and as a result, could not be applied to large batch sizes to be trained on a variety of input data. Thus, a batch size of 4 (in comparison to the suggested Iteration 1, which could handle 1024 without issue), was the limit of training.

Were this model implemented with pre-existing weights from being trained on datasets such as ImageNet (Deng et al., 2009), known as transfer learning, the outputs may have improved. Though, ImageNet is not trained on mel-spectrogram images, and may be unable to detect patterns in audio to the same extent as simpler models. Thus, it was concluded that models beyond the complexity of VGG-16, including ResNet-50, EfficientNet, and other model types, were not to be experimented with.

4.7. Final Model

After much trial and error, the final iteration of the model was developed. The below **Figure 4.3** demonstrates the architecture of this model.

Including additional layers was considered, however, given the higher number of convolution filters in this model in comparison to the initial iteration, it was deemed unnecessary, and resulted in lower accuracy regardless. Outside of this, the differences between this model and the first iteration are the inclusion of an extra dense layer after flattening, increasing the node count to 1024, and scaling the remaining 2 layers up by a factor of 4. Alongside this, batch normalisation was introduced in training, to ensure convolution layers did not overfit specific class weights. Finally, the first convolutional layer had a kernel size reduction to 3x3, instead of 5x3. This allowed for a more consistent kernel size that focuses on individual frequencies more carefully, with less of an emphasis from surrounding frequencies.

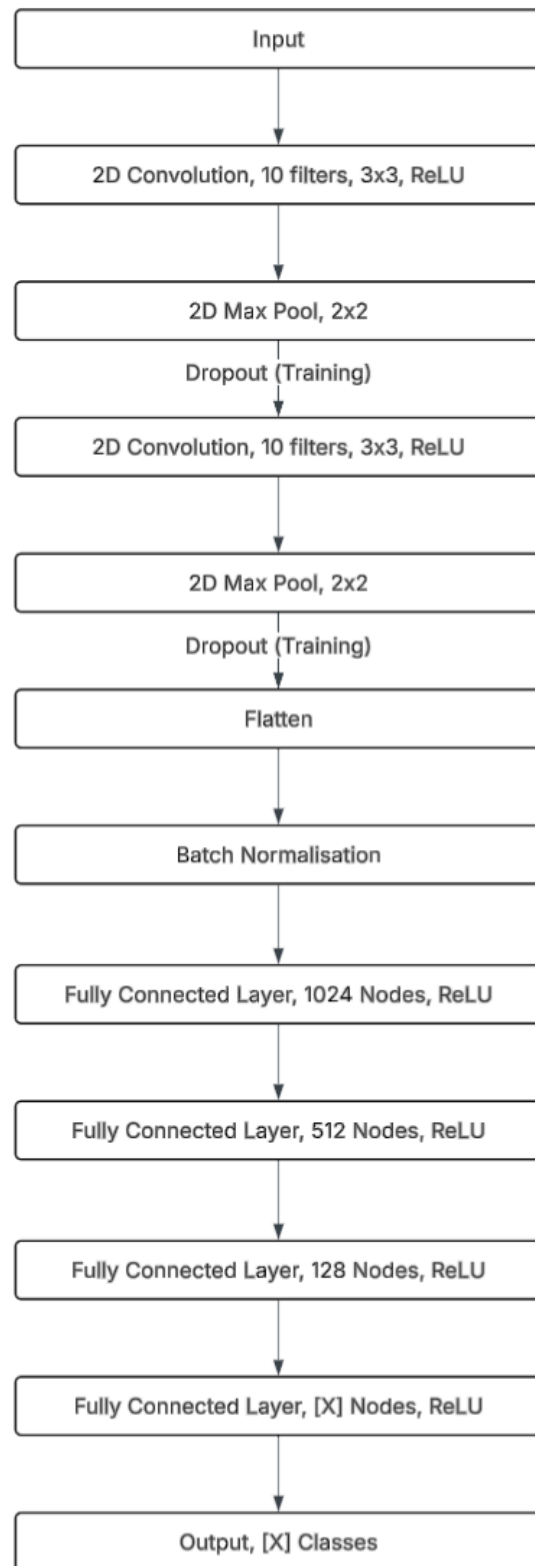


Figure 4.3 - Final model iteration

4.8. Summary

This chapter covered numerous areas of training the model, and a potential further development could be further experimentation with training order with specific species. For example, instead of ordering by genus alone, each species could be sorted by the total number of sound files in descending order. Thus, the first 50 species would have the largest wealth of training data, allowing for a potentially more accurate model. This, in comparison to a randomly generated sample, would have larger applicability, as these sound files would have a larger representation from the Xeno-Canto website, leading to a larger real-world representation. Were the data set smaller (10 classes, instead of 50), or a larger memory pool was available to run larger batch sizes, more complex models may have been able to be used with comparable results to that of the final model.

Chapter 5: Model Application, Testing and Results

5.1. Introduction

This chapter covers how the models performed on individual sound bytes, as well as sequential aggregation for applying the model on entire sound files.

5.2. Testing Strategy

The final iteration of the model was tested with the following hyperparameters:

- Sound sampling rate (Hz): 32,000
- File quality audio rating range: 5.0-5.0
- Classes: 50
- Sound byte duration (seconds): 2.0
- Sound files per class/folder limit: 20
- “Silent” identification variance: 0.2
- “Silent” identification mean decibel threshold: 5dB
- Train/test split: 80/20 respectively
- Sampling strategy: “median” (see Chapter 3.3)
- Loss function: categorical cross entropy loss
- Model optimiser: AdamW (Loshchilov & Hutter, 2017)
- Learning rate: 0.001
- Dropout rate: 0.5
- Batch size: 2,048

These were all chosen through rigorous testing and experimentation, seen in Appendix C. The best results for the model were found with these hyperparameters.

Before any model training commenced, random seeds were set to constant values such that all results were the direct result of hyperparameter tuning, and not random values leading to a local minima.

5.3. Sound Byte Results

All models were trained entirely on sound bytes, and as such, testing the model on the same object was the quickest method to evaluate the model’s performance. One major point worth noting is that sound bytes do not always represent an entire file. Many attempts were made to mitigate any errors as a result of this. These were discussed in the preprocessing techniques in Chapter 3.3.

Figure 5.1 shows the training and testing accuracy across 500 epochs of training using the strategy in Chapter 5.2.

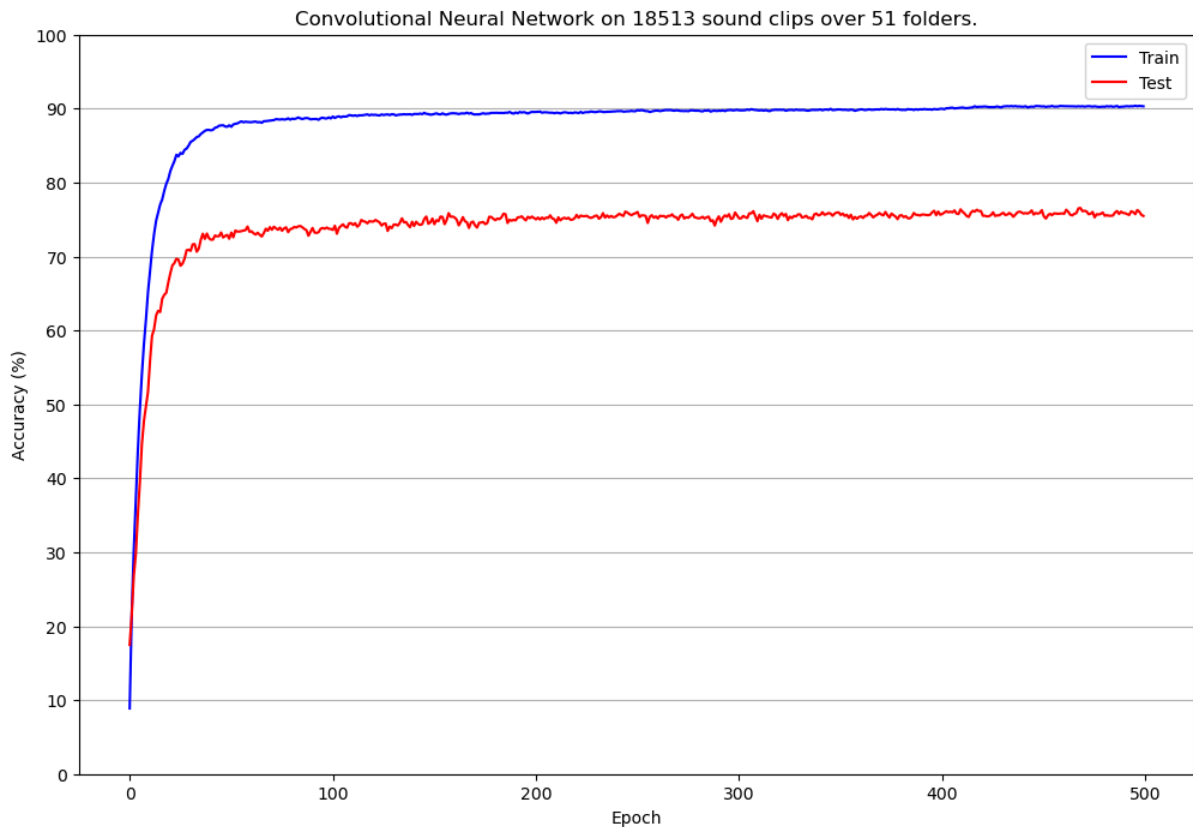


Figure 5.1 - CNN testing over 50 classes of sound bytes

It can be seen that the model very quickly reaches a peak of training and testing accuracy in less than 50 epochs, slowly increasing over time with minor variation. Concerns could be raised about continuing past a certain period of training, however, the model cannot overfit the data even after 500 epochs, only peaking at 90.28% training accuracy. This is due to the dropout regularisation in place, in which a proportion of nodes are not trained in a specific epoch. Similarly, the testing accuracy is roughly proportional to training accuracy, peaking at 76.56% on the 469th epoch.

Figure 5.1 mentions 51 classes, and this is because the 51st class is “silent” audio data, discussed in Chapter 3.3. Including silent audio sound bytes positively improved the accuracy as opposed to excluding it from training data. This can be seen in Appendix C, testing numbers 126 through 135, where silent audio is not its own trainable class. Across all 10 tests with variable variance thresholds, testing accuracy peaks at 75.74%, whereas the 136th test, where it is considered its own class, testing accuracy increases by 2.33%, to a final testing accuracy of 78.07%. This demonstrates that training a model on data that does not apply to any bird species helps reduce inaccurate predictions, and also highlights that some sound bytes across bird species may actually be “silent” data, but did not pass the “naive” threshold introduced.

The species-specific top-5 accuracy can be found in Appendix D. The average top-5 accuracy for sound bytes across every species is 87.84%, but when excluding the two species without any top-5 predictions, this jumps to 91.43%. This is very strong evidence to suggest that the model can accurately identify features of sound bytes that point towards a specific species.

5.4. Sequential Aggregation and Sound File Results

A further development beyond sound bytes is to classify entire files. There are many issues in training a model to classify bird species beyond sound bytes, as sound files can be in variable lengths, which models trained on specific sizes of parameters cannot adjust to.

Thus, the strategy to classify audio files is to split it into sound bytes of a specific length, classify each individual sound byte in the file, and then perform some aggregation function on these outputs. This strategy was implemented in (Noumida & Rajan, 2023). While it was not specified in this study which aggregation strategy was used, a variety of functions were tested in this research paper. Figure 5.2 below shows a demonstration of the aggregation strategy performed.

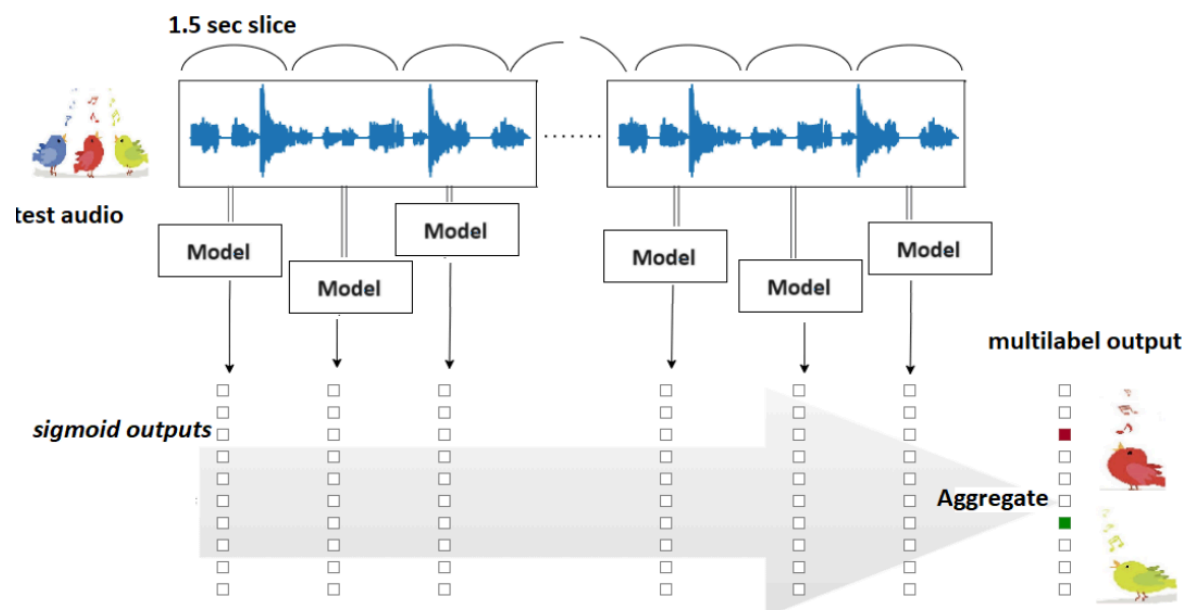


Figure 5.2 - Window sliding aggregation technique used to classify audio files (Noumida & Rajan, 2023, p. 1269)

Table 5.1 summarises results taken from analysing every audio file across the same 50 classes in Chapter 5.3.

Model iteration number	Aggregation function	Individual file accuracy	Overall class top-5 accuracy
Iteration 3 (final)	Sum outputs	1.75%	12.14%
Iteration 3 (final)	Normalise outputs, then sum	1.71%	11.45%
Iteration 2	Rank outputs, then sum. Lowest rank sum = stronger prediction	1.45%	9.87%
Iteration 3 (final)	Rank outputs, then sum. Lowest rank sum = stronger prediction	1.84%	9.40%

Table 5.1 - File classification and sequential aggregation function comparison

Table 5.1 shows many different results in aggregation. Other aggregation functions would have been tested, however, they all operate on a similar functional basis, such as applying logarithms and various exponents. For individual file accuracy, ranking outputs and summing the ranks, where lower rank sums are stronger predictions, appears to have a slightly better per-file accuracy as opposed to other techniques. However, it loses out strongly in top-5 accuracy, as ranking can drop off significantly as model outputs are not standardised across sound bytes. Normalisation was introduced to help combat this, and, in comparison to simply summing the outputs, it loses out slightly on both individual file accuracy and top-5 accuracy. As it was determined that simply summing the outputs produced the best results, this strategy was used for file classification in the final model.

However, the overall file accuracy is incredibly low. This accuracy is not acceptable for the model at all, and serves no practical purpose. It is statistically less likely to predict a file correctly than a model which chooses a species at random. It is slightly better than a random model in top-5 accuracy on a per-file basis, which is evidence of the model's ability to learn and adapt to different species, and correctly choose from many sound bytes.

Appendix E contains the species-specific file prediction results. It can be seen that certain species are not appearing whatsoever in any predictions.

Potential reasoning for this occurrence would be because of improper handling and treatment of data within the sound bytes. Various factors that affect model weights may not have been properly checked, as the "median" sampling may have not allowed minority classes to be factored for, while majority classes, with a wealth of sparse audio properties, direct the model to favouring those classes.

5.5. Summary

Despite the number of tests performed on model iterations, room for extra hyperparameter tuning could be implemented, but shows that the model has been trained extensively across 50 classes of audio files, but struggles to apply for audio file recognition on a whole. A further development from this would be to investigate silent sound bytes, as incorrect classifications of actual bird species may actually not have any bird calls present.

Chapter 6: Prototype Software Development

6.1. Introduction

This section covers the development timeline of a prototype desktop application to showcase the application of the developed model(s) on audio files. It will explore and compare various development environments, the user interface design, as well as a breakdown of the functionality of the program, with a discussion on the code implementation and future developments for it.

6.2. Development Environment and Framework

For this software, the Python language was chosen. The reasoning behind this language was due to the identification model itself being developed using Python libraries such as PyTorch (PyTorch Foundation, n.d.) and Pandas (NumFOCUS, n.d.). Developing this software in Python allows direct interactivity between the model and its outputs, and the user interface.

While other programming languages could provide a cleaner, professional, and complete look, such as Flutter (which runs the Dart language), with its extensive repertoire of “widgets” using these languages could run into compatibility issues with running the model (Flutter, 2026a). If this were the case, many solutions to this incompatibility would be to run the model on an external server, and have the client application transfer data to/from the server. Given the time constraints of this project, constructing an API while also updating the model over time for development changes would be too time consuming of a task. Thus, a fully client-side application was developed. This choice would drastically increase responsiveness and usability, supporting users without any internet connectivity.

A potential downside of using Python over environments such as Flutter is that multi-platform compatibility issues are common. Flutter, by default, has a large extent of platforms that are supported, including web applications, mobile devices, and desktop applications (Flutter, 2026b). Python does not have this range of platforms, without the use of external compilation software, which still does not guarantee multi-platform accessibility when implemented. Various packages and external tools exist to support the development of applications for other platforms, however, given the time scope of this project, testing these is not feasible.

6.3. User Interface

Within the Python language, there are numerous packages for developing UI applications. Common packages include:

- DearPyGui: a framework specialising in modern visualisation development.
- tkinter: a built-in Python module providing basic visualisation and interactivity.
- PyQt5: a module built for cross platform development, offering itself as an alternative to C++ program development.

- PySimpleGUI: a framework providing simplistic improvements and implementation over the above listed modules.

PyQt5 and DearPyGui are modules not necessarily specialised to the development of this prototype, and tkinter, while providing a strong advantage by not requiring external libraries, does not offer as streamlined an implementation as PySimpleGUI. Thus, PySimpleGUI was the framework chosen for development of the user interface.

As this software only serves to provide visual proof of the model's functionality, a very simple interface design is needed. The core components of this software are a dialog box to upload a local file, easily understandable buttons to execute the model, and a visual output for the model's predictions.

Figure 6.1 is a screenshot of the program, having run a prediction on a specific bird species.

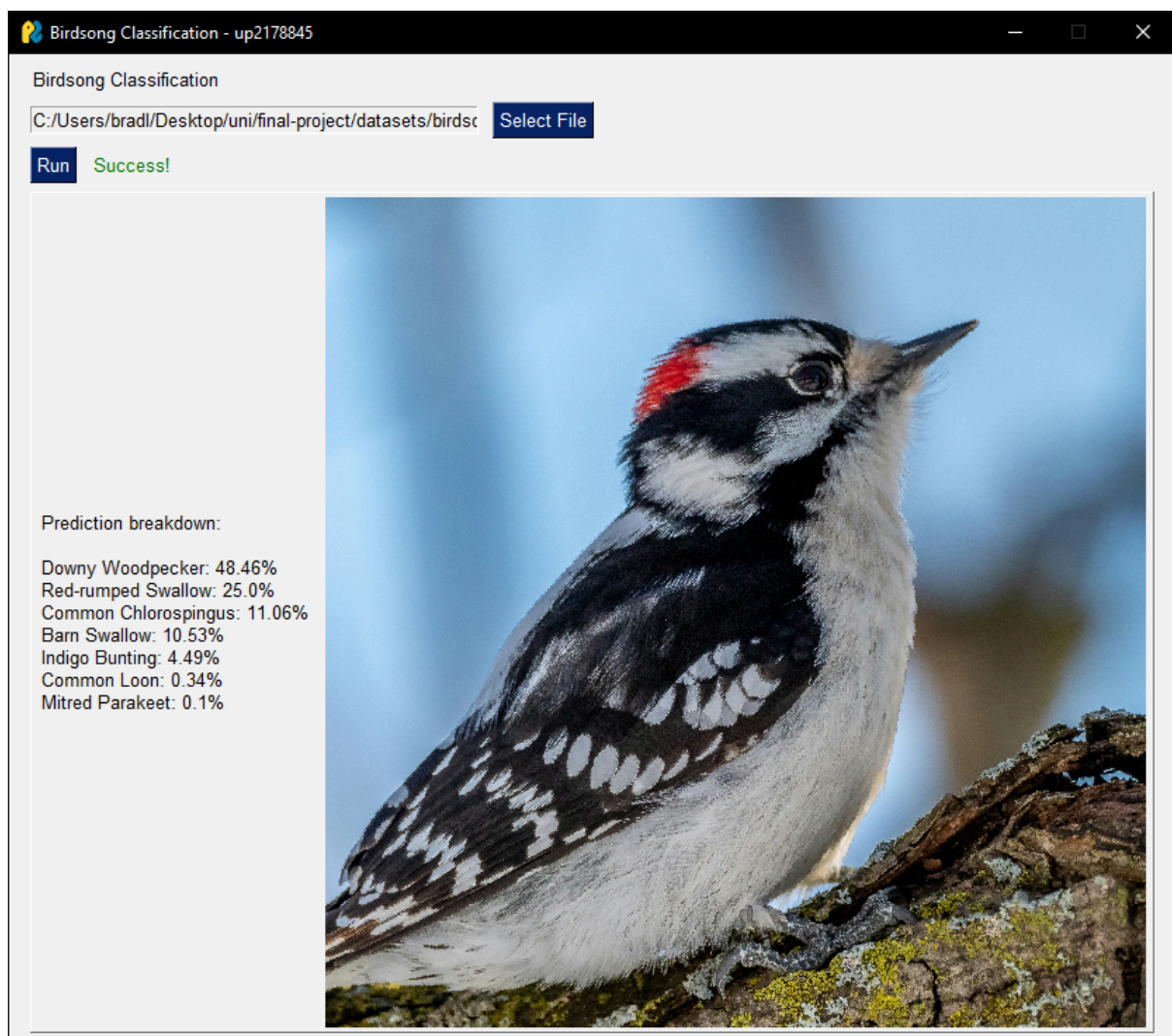


Figure 6.1 - Prototype software sample screenshot after running file successfully

The user interface is extremely simple, offering simple bird species prediction for the user. Upon startup, the user is prompted with 2 buttons, "Select File" and "Run". These are the only interactions that the user can perform. Upon selecting a file, pressing the "Run" button will run the algorithm, providing a detailed breakdown of the probabilities of a song being from a specific species. This is sorted in descending order, with an image of the most likely bird species shown alongside the predictions.

These images were taken from the Nuthatch API (Gnagy, n.d.), which contains bird species information, such as scientific names, habitat, conservation status, and (optionally) images. Some bird species do not have images, and thus, will not be displayed if this is the top prediction. All images, which were initially scraped from the Unsplash website, are completely free to download and use both commercially and non-commercially (Unsplash, 2026).

6.4. Program Functionality

The software functions efficiently, as there is not a lot that is required of the program. The flowchart below demonstrates typical user operation of the prototype software.

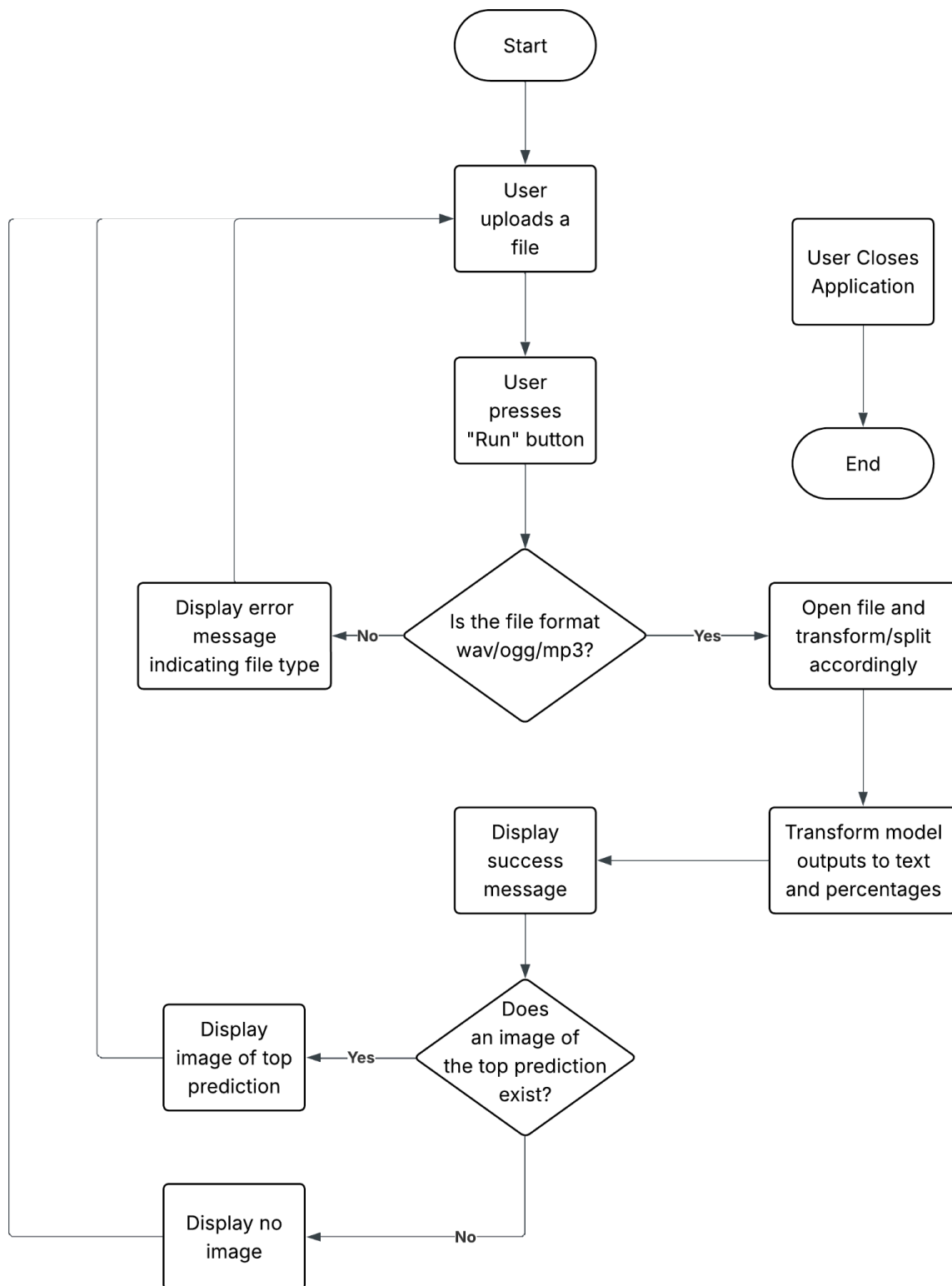


Figure 6.2 - Flowchart demonstrating program flow

6.5. Code Quality

Due to the nature of this software, as it is only a prototype artefact, code quality adhered lightly to common practices. Documentation was not included, as there is an

insufficient amount of code to the point where external usage is unlikely to occur. Code was commented where it was deemed necessary, with minor comments as to the function of each code block within the program. Code execution optimisation was not considered, primarily due to the time taken for file reading/writing taking up the majority of program execution time. There would be no seamless way to optimise how data is read by the external library that reads the files, thus it was left alone. There was no abstraction included in the code, as there were no classes that had any hierarchical dependencies. Finally, code was organised into functions for ease of reading for any external viewers. These helped identify within the program's main loop where and how specific functions would execute, allowing for ease of debugging.

6.6. Software Testing

The prototype software was not tested rigorously, largely again due to it being a demonstration artefact. Table 6.1 summarises potential error-inducing scenarios and how they were handled in the software. Since there are only 3 instances of user-interactivity within the software, the amount of possible user-induced errors is low.

Scenario	Expected Result	Solution
The user uploads an unsupported file type.	The external library fails to read the file's contents, raises an error, and the program crashes.	Check the file type before parsing the file to the external library, and alerting the user of supported file types.
The user uploads a wav/mp3/ogg file, but the path cannot be found/accessed.	The library attempts to read a file that doesn't exist and raises an error, crashing the program, or has insufficient permissions to read the file, causing another crash.	Check if the file exists, and alert the user if the file does not exist. Should not happen as the dialog box cannot be manually edited, and an existing file must exist.
The sound file, after transformation, contains zero useful sound bytes (pruning for silence/sound length too short).	The model is fed an array of length 0, and either raises an error, causing the program to crash, or returns zero(es).	Raise an error message before feeding the array, and alert the user the sound file contained no useful information.

Table 6.1 - Software testing case scenarios and solutions provided

6.7. Summary and Further Developments

The prototype software was developed within a short timespan, and as such, many attributes were not included, due to external time constraints and limited technical knowledge with the library.

Despite this, there are many developments that can be made to the software, and all would serve for an elevated user experience. The list below covers a variety of additional features:

- A graph of outcome probabilities. This would be displayed on a bar chart.
- A text summary of each/the top species. This would show information such as geographic location(s), wingspan, conservation status, etc.
- A sample noise of each predicted species. This would be a playable audio byte and could be taken from Xeno-Canto or other such files.
- Images of each species predicted. The software currently only shows the top species' image.
- A resizable window for dynamic displays. This would be a good step in the direction of multi-platform integration.
- Various extension features such as check boxes, accessibility features, etc.
- The ability to record and upload as a direct file. This would be a useful feature on mobile devices.
- Have the user train the model by asking if this was accurate or not. (Only possible if the user knows the species). This would be a large development as it could introduce extra species into the model's predictive capabilities. This also serves to ensure the model is trained on a lot of accurate data.
- External server integration. The model would be stored on a local server, and would constantly update with new information. This works hand in hand with the point listed above, to ensure all users have the best version of the model being trained.

Chapter 7: Conclusion

7.1. Aim & Objectives

There were many aims and objectives of this research, and a suitable number were met. The final major aim of this project, which was to develop a machine learning model capable of classifying at least 50 species of bird with an F1 score of at least 0.7, was unsuccessful. This was largely due to the model's incapability of aggregating sound bytes together to correctly classify an entire file, and serves as a lesson in further training a model to learn. If sufficient extra time were allocated to finding a way to train a model to learn after output aggregation, it may be very possible to produce a model with higher accuracy.

In regards to the objectives to meet this aim, all objectives were sufficiently met, with minor concern in regards to investigation of a variety of models on the dataset. Technical limitations, due to overfitting of the model on such a small batch size, combined with the volume of the models being unable to be stored and trained within memory, led to existing model architecture being insufficient for the dataset.

7.2. Risks

There were no major risks associated with the development of this model. Nothing had threatened the development of the software, outside of hardware restraints provided. The SCIAMA high computer cluster was being shared by other University students, meaning any one GPU node could have a multitude of users running equally complex computations in parallel. This significantly reduces the memory that can be allocated to running this task and storing any models in GPU memory. Despite this, models that were not as complex as ones such as VGG-16 could be stored in memory and applied to large batches of training data.

7.3. Project Management

The revised Gantt chart produced in Chapter 1 roughly followed the timeline indicated, with variations in task lengths. It did serve as a good indicator of the remaining time to allocate to certain tasks. For example, demo app development did not take nearly as much time as was given, as this was largely taken up by model finalisation, tweaking and tuning, as well as experimentation with other model architectures. Within each task, as the nature of machine learning model development is not linear, but instead cyclical, model development fell into a cyclical pattern of tweaking a model, training it, testing it, and comparing results to previous tweaks.

7.4. Further Development/Recommendations

There are a variety of further developments to be made from this research.

Firstly, the further development of a model that is capable of training itself on aggregations of sound bytes from a file, or modifying the existing model itself. This would be crucial as weights can be adjusted depending on previous results. A suitable technology for this would be a long short-term memory or a recurrent neural network model, as these models' architecture takes in previous outputs as inputs for deeper layers and works best on time-sensitive data.

Second, similar to what was performed in (Noumida & Rajan, 2023), a hybrid-network approach. This could be instanced from multiple neural networks being trained on a variety of audio file-specific information, such as standard spectrograms on a linear scale, location-sensitive information retrieved from metadata (to identify location-specific bird species) and mel frequency cepstrum coefficients (MFCCs), which are a much more compact attribute that can be fed into a model. Using this, a similar aggregate approach could be taken to generate a more accurate prediction for a bird species.

Another development would be to consider multiple models trained on varying lengths of sound bytes, similar to the above mentioned development. Instead of training multiple models on identical lengths, have models trained on varying sound byte lengths, which are then aggregated together. These could have an impact on model accuracy, due to the variety of bird calls where some birds have much longer lasting call patterns than other species.

The final model-related development that could be recommended would be to handle and transform the data with more precision. Due to the nature of how the audio was transformed into mel-spectrograms and limited technical knowledge with the libraries and mathematical knowledge on the subject, trimming audio bytes and transforming them such that only key audio is kept as training data would massively increase the predictive capability of the final model.

Outside of those listed in Section 6.7, the main further development for the prototype software would be a seamless integration between platforms, with a specific focus on mobile platforms. Mobile applications, as opposed to web or desktop applications, allow for the most convenient use and where those in nature would record bird sounds, having an application to efficiently identify a species to hand would improve the user experience of the model.

7.5. Conclusion

This research project showcases the development of a deep learning convolutional neural network model that can be trained and accurately predict bird species based on sound bytes of their calls, with further developments to be made in the realm of aggregation of sound bytes into whole audio files.

References

- Arduino. (n.d.). *Nano 33 BLE Sense | Arduino Documentation*. Nano 33 BLE Sense | Arduino Documentation. Retrieved 22 April 2026, from <https://docs.arduino.cc/hardware/nano-33-ble-sense/#tech-specs>
- Barrowclough, G. F., Cracraft, J., Klicka, J., & Zink, R. M. (2016). How Many Kinds of Birds Are There and Why Does It Matter? *PLOS ONE*, 11(11), e0166307. <https://doi.org/10.1371/journal.pone.0166307>
- Chawla, N. V. (2005). Data Mining for Imbalanced Datasets: An Overview. In O. Maimon & L. Rokach (Eds), *Data Mining and Knowledge Discovery Handbook* (pp. 853–867). Springer-Verlag. https://doi.org/10.1007/0-387-25465-X_40
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Choudhary, S., Karthik, C. R., Lakshmi, P. S., & Kumar, S. (2022). LEAN: Light and Efficient Audio Classification Network. *2022 IEEE 19th India Council International Conference (INDICON)*, 1–6. <https://doi.org/10.1109/INDICON56171.2022.10039921>
- Clements, J. F., P. C. Rasmussen, T. S. Schulenberg, M. J. Iliff, J. A. Gerbracht, D. Lepaga, A. Spencer, S. M. Billerman, B. L. Sullivan, M. Smith, & C. L. Wood. (2025). *The eBird/Clements checklist of Birds of the World: V2025* [Data set]. <https://www.birds.cornell.edu/clementschecklist/download/>
- Cornell Lab of Ornithology. (n.d.). *Cornell Lab of Ornithology Profile | Kaggle*. Cornell Lab of Ornithology Profile | Kaggle. Retrieved 29 April 2026, from <https://www.kaggle.com/organizations/cornelllabofornithology/competitions>

- Deng, J., Dong, W., Socher, R., Li, L.-J., Kai Li, & Li Fei-Fei. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255.
<https://doi.org/10.1109/CVPR.2009.5206848>
- Duan, L., Yang, L., & Guo, Y. (2024). SIAlex: Species identification and monitoring based on bird sound features. *Ecological Informatics*, 81, 102637.
<https://doi.org/10.1016/j.ecoinf.2024.102637>
- Duan, S., Towsey, M., Zhang, J., Truskinger, A., Wimmer, J., & Roe, P. (2011). Acoustic component detection for automatic species recognition in environmental monitoring. *2011 Seventh International Conference on Intelligent Sensors, Sensor Networks and Information Processing*, 514–519.
<https://doi.org/10.1109/ISSNIP.2011.6146597>
- Flutter. (n.d.-a). *Flutter—Build apps for any screen*. Google. Flutter - Build Apps for Any Screen. Retrieved 29 April 2026, from <https://flutter.dev/>
- Flutter. (n.d.-b). *Widget catalog*. Google. Widget Catalog. Retrieved 29 April 2026, from <https://docs.flutter.dev/ui/widgets>
- Fonseca, E., Favory, X., Pons, J., Font, F., & Serra, X. (2020). *FSD50K: An Open Dataset of Human-Labeled Sound Events* (Version 2). arXiv.
<https://doi.org/10.48550/ARXIV.2010.00475>
- Gairí, P., Pallejà, T., & Tresanchez, M. (2025). Environmental sound recognition on embedded devices using deep learning: A review. *Artificial Intelligence Review*, 58(6), 163. <https://doi.org/10.1007/s10462-025-11106-z>
- Gnagy, A. (n.d.). *Nuthatch API*. Last Elm Software. Nuthatch API. Retrieved 29 April 2026, from <https://nuthatch.lastelm.software/>

- He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition* (arXiv:1512.03385). arXiv.
<https://doi.org/10.48550/arXiv.1512.03385>
- Huang, Z., Tousnakhoff, A., Kozyr, P., Rehausen, R., Bießmann, F., Lachlan, R., Adjih, C., & Baccelli, E. (2024). TinyChirp: Bird Song Recognition Using TinyML Models on Low-power Wireless Acoustic Sensors. *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*, 1–10.
<https://doi.org/10.1109/IS262782.2024.10704131>
- Hyun Choi, D., Ha Joo, Y., Hong Kim, K., Ho Park, J., Joo, H., Kong, H.-J., Lee, H., Jun Song, K., & Kim, S. (2024). A Development of a Sound Recognition-Based Cardiopulmonary Resuscitation Training System. *IEEE Journal of Translational Engineering in Health and Medicine*, 12, 550–557.
<https://doi.org/10.1109/JTEHM.2024.3433448>
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2017). *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference* (arXiv:1712.05877). arXiv.
<https://doi.org/10.48550/arXiv.1712.05877>
- Ko, J., Kim, H., & Kim, J. (2022). Real-Time Sound Source Localization for Low-Power IoT Devices Based on Multi-Stream CNN. *Sensors*, 22(12), 4650.
<https://doi.org/10.3390/s22124650>
- Kumari, S., Roy, D., Cartwright, M., Bello, J. P., & Arora, A. (2019). EdgeL³: Compressing L³-Net for Mote Scale Urban Noise Monitoring. *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 877–884. <https://doi.org/10.1109/IPDPSW.2019.00145>

- Laksono, B. S. P., & Prasetyo, B. H. (2023). Speaker Recognition on Low Power Device Using Fully Convolutional QuartzNet. *Proceedings of the 8th International Conference on Sustainable Information Engineering and Technology*, 619–624. <https://doi.org/10.1145/3626641.3626946>
- Lin, T.-H., Chang, C.-T., Zhuang, T.-H., & Putranto, A. (2024). Real-time hollow defect detection in tiles using on-device tiny machine learning. *Measurement Science and Technology*, 35(5), 056006. <https://doi.org/10.1088/1361-6501/ad2665>
- Liu, Q., Wan, G., Zhang, Y., Zhong, G., Wang, L., & Liang, W. (2025). A novel method for the efficient capture of ground-nesting parent birds. *Ethology Ecology & Evolution*, 37(6), 800–807. <https://doi.org/10.1080/03949370.2025.2563573>
- Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization* (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.1711.05101>
- Maayah, M., Abunada, A., Al-Janahi, K., Ahmed, M. E., & Qadir, J. (2023). LimitAccess: On-device TinyML based robust speech recognition and age classification. *Discover Artificial Intelligence*, 3(1), 8. <https://doi.org/10.1007/s44163-023-00051-x>
- Mangalam Sankupellay, & Konovalov, D. (2018). *Bird Call Recognition using Deep Convolutional Neural Network, ResNet-50*. <https://doi.org/10.13140/RG.2.2.31865.31847>
- Montino, P., & Pau, D. (2019). Environmental Intelligence for Embedded Real-time Traffic Sound Classification. *2019 IEEE 5th International Forum on Research*

and Technology for Society and Industry (RTSI), 45–50.

<https://doi.org/10.1109/RTSI.2019.8895517>

Mou, A., & Milanova, M. (2024). Performance Analysis of Deep Learning Model-Compression Techniques for Audio Classification on Edge Devices. *Sci*, 6(2), 21. <https://doi.org/10.3390/sci6020021>

Nagrani, A., Chung, J. S., & Zisserman, A. (2017). VoxCeleb: A large-scale speaker identification dataset. *Interspeech 2017*, 2616–2620. <https://doi.org/10.21437/Interspeech.2017-950>

Noumida, A. K., & Rajan, R. (2021). Deep Learning-based Automatic Bird Species Identification from Isolated Recordings. *2021 8th International Conference on Smart Computing and Communications (ICSCC)*, 252–256. <https://doi.org/10.1109/ICSCC51209.2021.9528234>

Noumida, A. K., & Rajan, R. (2023). Multi-Label Bird Species Classification Using Sequential Aggregation Strategy from Audio Recordings. *Computing and Informatics*, 42(5), 1255–1280. https://doi.org/10.31577/cai_2023_5_1255

NumFOCUS. (n.d.). *pandas—Python Data Analysis Library*. Retrieved 29 April 2026, from <https://pandas.pydata.org/>

NVIDIA. (2021). *NVIDIA A100 | Tensor Core GPU*. NVIDIA. NVIDIA A100 | Tensor Core GPU. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf>

Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. *Proceedings of the 23rd ACM International Conference on Multimedia*, 1015–1018. <https://doi.org/10.1145/2733373.2806390>

PyTorch Foundation. (n.d.). *PyTorch*. PyTorch - PyTorch. Retrieved 29 April 2026, from <https://pytorch.org/projects/pytorch/>

RSPB. (n.d.). *Corvids: How to identify the crow family*. RSPB. Corvids: How to Identify the Crow Family. Retrieved 29 April 2026, from <https://www.rspb.org.uk/birds-and-wildlife/identifying-birds/corvids-how-to-identify-the-crow-family>

Shearer, C. (2000). The CRISP-DM Model: The New Blueprint for Data Mining. *Journal of Data Warehousing*, 5, 13–22.

Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition* (arXiv:1409.1556). arXiv. <https://doi.org/10.48550/arXiv.1409.1556>

Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. (2016). *Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning* (arXiv:1602.07261). arXiv. <https://doi.org/10.48550/arXiv.1602.07261>

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). *Rethinking the Inception Architecture for Computer Vision* (arXiv:1512.00567). arXiv. <https://doi.org/10.48550/arXiv.1512.00567>

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946). arXiv. <https://doi.org/10.48550/arXiv.1905.11946>

Unsplash. (2026). License | Unsplash. In *License | Unsplash*. Unsplash. <https://unsplash.com/license>

Wang, J.-J., Sharma, A. K., Liu, S.-H., Zhang, H., Chen, W., & Lee, T.-L. (2024). Prediction of Vascular Access Stenosis by Lightweight Convolutional Neural

Network Using Blood Flow Sound Signals. *Sensors*, 24(18), 5922.

<https://doi.org/10.3390/s24185922>

Wyatt, S., Elliott, D., Aravamudan, A., Otero, C. E., Otero, L. D., Anagnostopoulos, G. C., Smith, A. O., Peter, A. M., Jones, W., Leung, S., & Lam, E. (2021). Environmental Sound Classification with Tiny Transformers in Noisy Edge Environments. *2021 IEEE 7th World Forum on Internet of Things (WF-IoT)*, 309–314. <https://doi.org/10.1109/WF-IoT51360.2021.9596007>

Xeno-canto Foundation. (n.d.). *Frequently Asked Questions: Xeno-canto*.

Xeno-canto Foundation. Frequently Asked Questions :: Xeno-Canto.

Retrieved 29 April 2026, from <https://xeno-canto.org/help/FAQ>

Appendices

Appendix A: Project Initiation Document



Project Initiation Document

Project Details:

Project Title	Using neural networks to classify bird species from songs and calls
Student	Bradley Smith
Course	Data Science and Analytics
Project code	PJE40
Client	N/A
Supervisor	Farzad Arabikhan
Date	September 2025

1. Client / Target Audience

There is no one specific client for this project, however the target audience for the development of this project would be people who have a mild interest in birding and/or those interested in using neural networks for classification of audio files.

A potential further target audience would be software developer(s) who would be developing an application to help identify birds based on a specific call.

2. Project background and problem to be solved

In the modern world, many people are disconnected from nature, with the fast growth of technology and rise in convenience. When people take a step back and try to re-establish their roots with nature, one common way is to take a walk in nature. The most common sound people hear are the sounds of many birds calling. However, these birds are impossible to notice, be it due to their size, speed, or even their timid nature, leading people to never actually see these birds. As a result, one way people are able to properly identify these birds is through “banding” these birds, by catching them in nets, potentially causing unnecessary stress to the birds, especially if just for the purpose of identification of a species. My goal is to reduce this approach to identifying birds, saving birding enthusiasts/organisations lots of time, effort and resources, allowing them to identify birds in a way that causes minimal stress to these birds.

3. Project aim and objectives

The final goal of this project is to have a machine learning model that will accurately classify a testing set of bird calls and songs.

The aim of this project and the necessary objectives are:

- Create a machine learning model that can classify over 100 species of bird with an F1 score of at least 0.7. This is because of the Kaggle dataset leaderboards showing performance metrics in the ranges of 0.65-0.8 for F1 scores.
- If the above is not possible, create a machine learning model that can classify up to 100 species of bird with an F1 score of at least 0.9. This is due to having a smaller number of species to classify.
- To do these, I will:
 - Create multiple types of machine learning models and compare the performance metrics of each.
 - Combine data from existing BirdCLEF and Cornell Lab of Ornithology datasets and collate the 5 datasets from 2020-2024 to investigate a larger range of data.
 - I will not include any duplicate audio files to skew learning capabilities and reduce redundancy.

4. Project constraints

One constraint for the project will be the spectrum of bird species that the model is able to classify. If all the datasets that my model is trained on do not include calls from a specific species of bird, then using any audio recordings of said species of bird will be pointless.

One limiting factor for this project is also that there is no budget. If there are any audio recordings or databases locked behind a one-time payment, or a subscription, that data will not be available for analysis, even if it could have been vital for this project's potential.

One potential constraint would be the combination of technology that the machine learning model is developed on, as well as the deadline of the project. If the model is trained on a less powerful machine, then the time taken to fully train the model over one epoch could take considerably longer. The fastest method of training a complex ML model - GPU acceleration - can only be as fast as the most powerful workstation available to me. Given the deadline of the project, any time I need to fine-tune the model to produce a single output could change from minutes per epoch, up to hours (if the model is complex enough.)

5. Project management

For my project, the Agile methodology seems to best apply to the development of my model. However, I will be also planning stages of development which will, within themselves, have Agile development aspects. I will be splitting the project into phases of development, reiterating each phase until I am satisfied, and only once I am satisfied with the end result, I move on to the next phase. This will likely be split up into sections such as: data collection, data cleaning, data preprocessing, machine learning model development, and finally model testing. With the final two phases, I will go back and forth between the two to ensure my model accurately meets testing criteria, fine-tuning the model each time testing results are not accurate enough. I aim to dedicate a minimum of 2 weeks for each phase, with the final two phases requiring most of my time.

6. Tasks and timescales

No	Stage	Dates	Main Tasks
1	Project Initiation	22nd Sept 2025 - 1st Oct 2025	Establishing supervisor and submitting PID.
2	Literature Review	22nd Sept 2025 - 19th Dec 2025	Reading and critically reviewing all relevant literature to the subject.
3	Ethics Form Application	8th Oct 2025 - 5th Nov 2025	Writing and submitting ethics form.
4	Satisfactory Progress Presentation	17th Nov 2025 - 8th Dec 2025	Establishing development progress for the model.
5	Field Data Collection	10th Nov 2025 - 25th Feb 2026	Collecting audio recordings of wild birds to test any/all models.
6	Dataset Handling	6th Oct 2025 - 24th Oct 2025	Procuring, cleaning and exploring any relevant datasets.
7	Model Prototype	24th Oct 2025 - 20th Nov 2025	Developing a small-scale model prototype as proof of concept.
8	First Full Model Iteration	24th Nov 2025 - 15th Dec 2025	Develop a full-scale model to test and assess capabilities.
9	Critical Analysis #1	15th Dec 2025 - 19th Dec 2025	Thorough analysis of all previous work mentioned above, detailing any necessary changes to be made.
10	Second Model Iteration	22nd Dec 2025 - 12th Jan 2026	Redevelopment of the model following critical analysis #1. Training and re-testing.
11	Critical Analysis #2	12th Jan 2026 - 16th Jan 2026	Analysis of differences between first and second iteration of models, potential investigation into other machine learning models.
12	Further Developments	19th Jan 2026 - 20th Mar 2026	Repeats of steps 10-11 or complete redevelopment of the model if applicable until a final model is established and tested.
13	Report Submission	19th Jan 2026 - 6th May 2026	Writing of dissertation up to and including submission.

7. Facilities and resources

For this project, there are various pieces of hardware and software that I will need to complete this to the best of my ability.

To facilitate the flexibility of my working locations, any machines should have Git, a version control architecture that syncs workstations to allow for up-to-date progress on any machine that is connected to a repository.

All the software/libraries I plan to use are available on all major operating systems (Windows, Mac, Linux), so operating system differences are not an issue.

In terms of hardware requirements, I require a computer that has capability of developing machine learning models, ideally with the use of TensorFlow, which is a library which can utilise a machine's GPU to greatly accelerate learning/training of a machine learning model.

- I have constant access to a PC at my home that satisfies these requirements. The hardware specification is as follows:
 - Memory: 32GB DDR4-2133
 - CPU: Intel Core i9-9900k (8 cores, 16 threads)
 - GPU: NVidia GeForce RTX 3060
 - Storage: 2TB NVME M.2 drive, 1TB NVME M.2 drive, 1TB SATA HDD, totalling 4TB of total storage.
- When I am not at home, development can be resumed via PCs at the University, where the hardware specifications of workstations are unknown, due to the variation among locations containing PCs. These PCs are not available 24/7, due to the buildings closing overnight.

In regards to datasets, I will be mainly training/testing my model on the Cornell Lab of Ornithology competition datasets on Kaggle. The below table describes the datasets.

<https://www.kaggle.com/organizations/cornelllabofornithology/competitions>

Dataset/Competition	Dataset Size (GB)	Total Audio File Count	Total Unique Classes
Cornell Birdcall Identification: https://www.kaggle.com/competitions/birdsong-recognition	25.35	21,375	264
BirdCLEF 2021 - Birdcall Identification: https://www.kaggle.com/competitions/birdclef-2021	42.18	62,874	397
BirdCLEF 2022: https://www.kaggle.com/competitions/birdclef-2022	6.61	14,852	152
BirdCLEF 2023: https://www.kaggle.com/competitions/birdclef-2023	5.29	16,941	264
BirdCLEF 2024: https://www.kaggle.com/competitions/birdclef-2024	23.43	24,459	182
Combined (Removing duplicates, best approximation)	75.77	112,891	992

8. Project risks

No	Description	Likelihood	Impact	Mitigation/Avoidance
1	Storage failure	Low	Loss of data for the project on a given system, resulting in hours/days worth of time loss.	Use GitHub to store progress in the Cloud, and keep physical copies on multiple systems.
2	Unavailability of participants to procure testing data.	Medium	Lack of fresh testing data reduces real world applicability of the model. Potentially affecting final testing accuracy.	Personally seek testing data where possible, or find files online of already-classified birds to test the model on.

9. Project deliverables

Outside of my final report document, other files that will be delivered are any procured audio files by myself or participants, Python notebook(s) containing the process of developing this model. There will also be a file for each phase's model iteration, stored in a file format native to TensorFlow.

There will also be anonymised documents from each participant stating where and when they recorded the audio files, for how long, and any additional notes to add to these recordings.

10. Research

I will do secondary research on many topics, including the ethics/safety of bird catching, whether any psychological stress is put on these birds who get caught in such nets, as well as other methodologies to identify birds without physically catching and temporarily restraining them.

For my model development, I will also do secondary research on data analysis on audio files, neural network optimization techniques with big datasets, as well as appropriate methodologies for classification analysis with hundreds of output nodes.

I do not plan to do any primary research.

11. Legal, ethical, professional, social issues

Legal:

- When myself or participants are recording any audio files for classification analysis, we will ensure we adhere to all laws regarding trespassing, nature reserves, as well as ensuring all audio files are recorded for public use, and do not contain any third party human voices, as there could be a potential violation of audio file usage without any third party's permission.

Ethical:

- One potential ethical issue with this project is that when recording any audio files of birds, we must take care not to cause any psychological stress to the birds or any other wildlife in the area, as the aim of this project is to identify birds without causing any stress at all.
- When storing information on the audio files, any and all information must be handled such that any sensitive information cannot be obtained, such as extremely detailed location information, and that all participant information is anonymised such that they cannot be identified from the files.

Professional:

- Any data or information submitted in my deliverables must be fully secure and not exploitable or contain malicious code. I aim to source all of my data from verifiable sources, and thoroughly check for any abnormalities.
- One problem is the accuracy of individually sourced data of bird songs. Unless we already know which bird is making the call, it would be difficult/impossible to determine if the model can accurately predict which bird is making the call. If this is the case, professional birders will be consulted and will help classify the bird call. If this is not possible, the data will not be used for testing.

12. Supervisor meetings

Upon discussion with my supervisor, we plan to have weekly meetings at 12:30pm on Fridays, in person. If it is not possible to have the meetings in person, they could be rescheduled or they will be on a video call instead.

13. Declarations

Please tick the following declarations:

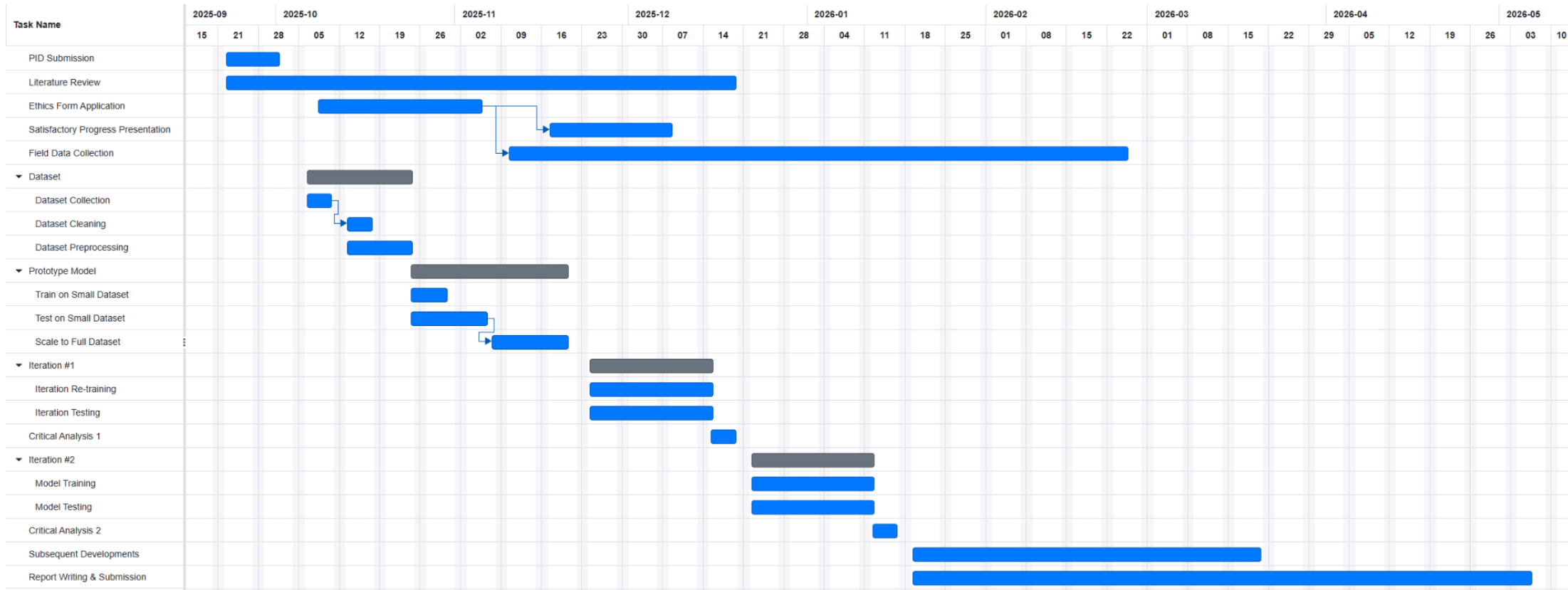
Usage (optional)	
☒	I give permission for this document to be made available to other students as examples of previous work.
Authenticity of work	
☒	I confirm that I have read the University rules in respect of plagiarism and student misconduct.
☒	I understand that if I use work from an external source, I must reference and cite the source in any work that I produce.
Ethics	
☒	I understand that ethical approval will be needed for this project regardless of whether I carry out primary research or not.
☒	I understand that if I do conduct primary research, I must include all raw data and research documentation in my final project report.
☒	I understand that not including the raw data can mean potentially failing the project.
☒	I understand that not obtaining ethical approval is grounds for academic misconduct and possible project failure.
Use of AI Artificial Intelligence (AI), such as ChatGPT, can be used as a tool to help assist and inform the initial development of your work but it is not a replacement for your own critical thinking and analysis. We require that your work is your own original content, demonstrating your knowledge, skills, and critical thinking abilities. To that end, AI-generated content must not be included in <u>any work</u> you submit for assessment unless suitably referenced. Not doing so is a breach of the University's Academic Regulations as outlined in the Student Conduct Policy and constitutes an assessment offence.	
☒	I have read the University AI policy and if relevant, I will acknowledge the use of AI in any work I produce for this project with appropriate referencing.
☒	I understand that not appropriately referencing the use of AI is an assessment offence equal to plagiarism and may result in a penalty which remains on my student record.

Name: Bradley Smith

Date: 23rd September 2025

Appendix: Gantt chart

The following is my project development timeline. The full image can be found in [my GitHub repository](#). There is also a “.gantt” file which can be opened online in the [Online Gantt](#) website.



Appendix B : Ethics Certificate



Certificate of Ethics Review

Project title: Classification of bird species from calls using deep learning methods

Name:	Bradley Smith	User ID:	2178845	Application date:	03/11/2025 20:52:40	ER Number:	TETHIC-2025-112043
-------	---------------	----------	---------	-------------------	------------------------	------------	--------------------

You must download your referral certificate, print a copy and keep it as a record of this review.

The FEC representative(s) for the **School of Computing** is/are [Elisavet Andrikopoulou](#), [Kirsten Smith](#)

It is your responsibility to follow the University Code of Practice on Ethical Standards and any Department/School or professional guidelines in the conduct of your study including relevant guidelines regarding health and safety of researchers including the following:

- [University Policy](#)
- [Safety on Geological Fieldwork](#)

It is also your responsibility to follow University guidance on Data Protection Policy:

- [General guidance for all data protection issues](#)
- [University Data Protection Policy](#)

Which school/department do you belong to?: **School of Computing**

What is your primary role at the University?: **Undergraduate Student**

What is the name of the member of staff who is responsible for supervising your project?: **Farzad Arabikhan**

Is the study likely to involve human subjects (observation) or participants?: No

Will financial inducements (other than reasonable expenses and compensation for time) be offered to participants?: No

Are there risks of significant damage to physical and/or ecological environmental features?: No

Are there risks of significant damage to features of historical or cultural heritage (e.g. impacts of study techniques, taking of samples)?: No

Does the project involve animals in any way?: Yes

Could the research outputs potentially be harmful to third parties?: No

Could your research/artefact be adapted and be misused?: No

Will your project or project deliverables be relevant to defence, the military, police or other security organisations and/or in addition, could it be used by others to threaten UK security?: No

Please read and confirm that you agree with the following statements: I confirm that I have considered the implications for data collection and use, taking into consideration legal requirements (UK GDPR, Data Protection Act 2018 etc.), I confirm that I have considered the impact of this work and and taken any reasonable action to mitigate potential misuse of the project outputs, I confirm that I will act ethically and honestly throughout this project

Supervisor Review

As supervisor, I will ensure that this work will be conducted in an ethical manner in line with the University Ethics Policy.

Supervisor comments:

Supervisor's Digital Signature: **farzad.arabikhan@port.ac.uk** Date: **14/11/2025**

Faculty Ethics Committee Review

Ethics Rep comments: **Only secondary data use.**

Faculty Ethics Committee Member's Digital Signature(s): **elisavet.andrikopoulou@port.ac.uk** Date: **05/02/2026**

Appendix C: Full Test Result Timeline

Note: some columns were omitted and row values were shortened due to space constraints and ease of readability. The full spreadsheet of results can be found at ([GITHUB LINK](#))

test #	model #	# classes	single or multi label	file rating range	sample rate kHz	sound length (secs)	files per folder threshold	sound clips per file	variance threshold	mean db threshold	test size	sampling	sound clips	epochs	optimizer	learning rate	dropout rate	batch size	train accuracy %	test accuracy %	gpu train time secs	top-5 accuracy (%)	other changes
1	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.001	0	64	100	61	39	N/A	
2	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.01	0	64	100	52	39	N/A	
3	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.001	0.3	64	98	68	39	N/A	
4	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.001	0.2	64	98	73	39	N/A	
5	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.01	0.2	64	98	52	39	N/A	
6	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.001	0.1	64	12	11	39	N/A	
7	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.01	0.1	64	100	63	39	N/A	
8	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.001	0.5	64	12	11	39	N/A	
9	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.01	0.5	64	12	11	39	N/A	

10	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.1	0.5	64	12	11	39	N/A	
11	1	13	multi	0.0-5.0	32	2	0	100	N/A	N/A	0.2	none	1000-1300	500	SGD	0.005	0.25	64	98	68	39	N/A	
12	1	35	multi	0.0-5.0	32	2	0	50	N/A	N/A	0.2	none	1000-1300	500	SGD	0.005	0.25	64	98	47	73	N/A	
13	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.005	0.25	64	90	48	75	N/A	
14	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.01	0.15	64	11	10	75	N/A	
15	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.001	0	64	90	48	75	N/A	
15	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.01	0.25	16	100	55	95	N/A	
16	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.01	0.5	16	100	42	95	N/A	
16	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.2	none	2145	500	SGD	0.001	0.15	64	100	66	75	N/A	
17	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	500	SGD	0.01	0.5	16	100	52	95	N/A	
18	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.001	0.5	16	100	56	33	N/A	
19	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.001	0.5	64	94	61	25	N/A	
20	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.001	0.5	128	85	67	25	N/A	
21	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.001	0.5	256	66	59	25	N/A	
22	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.001	0.5	32	94	65	27	N/A	
23	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.01	0.5	32	100	38	26	N/A	
24	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.005	0.5	32	100	53	25	N/A	
25	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.0001	0.5	32	70	60	26	N/A	
26	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	200	SGD	0.0001	0.2	32	36	36	27	N/A	
27	1	13	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	2145	1000	SGD	0.0001	0.2	32	98	62	126	N/A	

28	1	25	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	4717	1000	SGD	0.0001	0.2	32	93	32	297	N/A	
29	1	25	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	4717	1000	SGD	0.001	0.2	32	100	34	297	N/A	
30	1	25	multi	0.0-5.0	32	2	0	200	N/A	N/A	0.3	none	4717	1000	SGD	0.001	0.5	64	100	30	270	N/A	
31	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	1000	SGD	0.001	0.5	64	67	44	967	N/A	changed method to look at 10 full folders. tested on LAPTOP. Peaked at 53% test acc.
32	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	200	SGD	0.01	0.5	32	58	38	252	N/A	
33	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	200	SGD	0.01	0	32	30	30	242	N/A	
34	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	200	SGD	0.01	0	64	63	40	185	N/A	
35	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	100	SGD	0.01	0	128	56	41	100	N/A	
36	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	100	SGD	0.001	0	128	60	54	100	N/A	
37	1	10	multi	0.0-5.0	32	2	0	0	N/A	N/A	0.3	none	14922	300	SGD	0.001	0	128	63	49	190	N/A	
38	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	none	8959	100	SGD	0.001	0	128	72	60	50	N/A	filtered by looking at 50-100 files per folder
39	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	under	3520	300	SGD	0.001	0	128	96	56	57	N/A	applies random undersampling from files (deletes random ones from all classes except minority)
40	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	22140	500	SGD	0.001	0	128	81	66	624	N/A	applies random oversampling from files (duplicates random rows from all classes except majority)
41	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	22140	500	SGD	0.001	0	64	87	68	630	N/A	
42	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	18480	500	SGD	0.001	0	64	84	67	515	N/A	first instance of saving best model to file. after this, the best testing accuracy is saved. alongside the corresponding training accuracy
43	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	13560	500	SGD	0.001	0	64	83	59	523	92.80	top 5 error introduced

44	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	22140	100	SGD	0.001	0	64	83	70	140	N/A	
45	1	25	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	70675	100	SGD	0.001	0	64	60	51	393	81.87	
46	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	69250	100	SGD	0.001	0	64	43	35	389	68.72	
47	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	69250	100	SGD	0.01	0	64	48	38	393	76.03	
48	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	69250	100	SGD	0.01	0	32	38	30	400	64.52	
49	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	69250	100	SGD	0.01	0	128	38	34	721	72.21	
50	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	70325	100	SGD	0.01	0	16	39	33	546	70.97	
51	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	70325	100	SGD	0.001	0	16	48	37	541	73.86	
52	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	70325	100	Adam	0.001	0	16	47	38	1050	76.69	first switch to adam optimizer. DOES use more memory than SGD but provides better results
53	1	25	multi	0.0-5.0	32	2	[100, 150]	0	N/A	N/A	0.3	over	70325	100	AdamW	0.001	0	16	42	35	592	71.64	usage of adamW due to convergence and separating of weight decay and parameter updates
54	1	25	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	70725	100	AdamW	0.001	0	16	53	47	604	78.76	
55	1	10	multi	0.0-5.0	32	2	[50, 100]	0	N/A	N/A	0.3	over	7300	100	AdamW	0.001	0	16	84	52	45	87.44	first ever test using SCIAMA.
56	1	10	multi	0.0-5.0	32	2	[0, 0]	0	N/A	N/A	0.3	over	10090	100	AdamW	0.001	0	16	96	77	61	96.30	
57	1	10	multi	0.0-5.0	32	2	[0, 0]	0	N/A	N/A	0.3	over	10090	100	AdamW	0.001	0	128	98	77	13	96.20	
58	1	10	multi	0.0-5.0	32	2	[0, 0]	0	N/A	N/A	0.3	over	10090	500	AdamW	0.001	0	-1	93	76	48	97.46	

59	1	25	multi	0.0-5.0	32	2	[1, 50]	0	N/A	N/A	0.3	over	8875	500	AdamW	0.001	0	64	99	80	88	89.07	
60	1	10	multi	0.0-5.0	32	2	20	0	N/A	N/A	0.3	under	2713	100	AdamW	0.001	0	16	94	67	104	93.38	used TorchAudio instead of Librosa to load until next blue "other changes", all sound clips totals are BEFORE sampling
61	1	10	multi	0.0-5.0	32	2	50	0	N/A	N/A	0.3	under	7461	100	AdamW	0.001	0	16	98	49	29	82.72	
62	1	25	multi	0.0-5.0	32	2	50	0	N/A	N/A	0.3	under	17666	100	AdamW	0.001	0	16	94	36	70	56.65	
63	1	50	multi	0.0-5.0	32	2	50	0	N/A	N/A	0.3	under	34000	100	AdamW	0.001	0	16	84	21	66	35.47	
64	1	50	multi	0.0-5.0	32	2	50	0	N/A	N/A	0.3	under	34000	100	AdamW	0.001	0	64	86	22	23	38.09	
65	1	50	multi	0.0-5.0	32	2	50	0	N/A	N/A	0.3	under	34000	500	AdamW	0.001	0	64	98	22	115	35.29	
66	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.3	under	34000	500	AdamW	0.001	0	64	56	21	343	43.64	
67	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.2	under	66124	500	AdamW	0.001	0	64	54	22	352	43.21	
68	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.2	median	58000	500	AdamW	0.001	0	64	37	26	1154	53.26	introduction of "median" sampling. "sound clips" is now the total sound clips AFTER sampling.
69	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.3	under	30000	500	AdamW	0.001	0	64	46	23	499	46.87	
70	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.2	median	63750	500	AdamW	0.001	0	64	33	23	1175	50.28	
71	1	50	multi	0.0-5.0	32	2	100	0	N/A	N/A	0.2	median	65650	500	AdamW	0.001	0	64	33	24	1213	50.94	
72	1	10	single	4.0-5.0	32	2	0	0	N/A	N/A	0.2	median	4820	500	AdamW	0.001	0	64	100	81	83	95.85	first test after filtering for 4.0-5.0 audio file quality. Removed all classes with < 10 files after filtering. also removed ALL multi label files
73	1	25	single	4.0-5.0	32	2	0	0	N/A	N/A	0.2	median	12300	500	AdamW	0.001	0	64	100	66	209	82.52	
74	1	50	single	4.0-5.0	32	2	100	0	N/A	N/A	0.2	median	26200	500	AdamW	0.001	0	64	69	46	465	66.76	
75	1	100	single	4.0-5.0	32	2	100	0	N/A	N/A	0.2	median	45100	500	AdamW	0.001	0	64	46	35	781	56.44	

76	1	100	single	4.0-5.0	32	2	50	0	N/A	N/A	0.2	median	41900	500	AdamW	0.001	0	64	52	38	736	60.00	
77	1	200	single	5.0-5.0	32	2	0	0	N/A	N/A	0.2	median	108400	500 (48)	AdamW	0.001	0	64	26	22	990	40.49	introduction of experimental epoch training. stops training after 5 consecutive epochs of no updates to testing accuracy.
78	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	48800	500	AdamW	0.001	0	64	44	32	865	54.95	
79	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (37)	AdamW	0.001	0	64	34	27.32	207	50.68	set order of classes to analyse. Look at folder_order.txt for more details
80	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (47)	AdamW	0.001	0	128	34.95	28.02	228	51.36	
81	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (51)	AdamW	0.001	0	256	35.33	27.53	227	50.72	
82	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (81)	AdamW	0.001	0	1024	36.54	27.82	529	50.93	
83	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (69)	AdamW	0.001	0	1024	35.06	27.18	447	49.93	shuffling of training data every epoch
84	1	100	single	5.0-5.0	32	2	100	0	N/A	N/A	0.2	median	66400	500 (94)	AdamW	0.001	0	1024	36.63	28.2	239	51.04	shuffling of batch data every epoch
85	1	100	single	5.0-5.0	32	2	0	0	N/A	N/A	0.2	median	66400	500 (59)	AdamW	0.001	0	1024	36.09	27.12	155	49.73	
86	1	100	single	4.0-5.0	32	2	0	0	N/A	N/A	0.2	median	125500	500 (60)	AdamW	0.001	0	1024	29.42	24.51	352	46.06	
87	1	50	single	5.0-5.0	32	2	0	0	N/A	N/A	0.2	median	31200	500	AdamW	0.001	0	1024	63.2	40.61	299	64.50	
88	1	50	single	5.0-5.0	32	2	0	0	N/A	N/A	0.2	median	31200	500	AdamW	0.001	0	64	57.44	39.54	671	64.12	
89	1	50	single	5.0-5.0	32	2	0	0	N/A	N/A	0.2	median	31200	500	AdamW	0.001	0	512	62.04	40.53	317	63.90	
90	1	50	single	5.0-5.0	32	1.5	0	0	N/A	N/A	0.2	median	42200	500	AdamW	0.001	0	512	46.57	34.53	371	59.24	
91	1	50	single	5.0-5.0	32	1	0	0	N/A	N/A	0.2	median	63850	500	AdamW	0.001	0	512	34.71	29.49	515	55.09	

92	1	50	single	5.0-5.0	32	2.5	0	0	N/A	N/A	0.2	median	24950	500	AdamW	0.001	0	512	81.75	46.19	258	65.93	
93	2	50	single	5.0-5.0	32	2.5	20	0	N/A	N/A	0.2	median	15600	500 (23)	AdamW	0.001	0	128	90.47	63.72	62	78.62	new model. Batch normalisation applied + other features (check diff between models in model_iterations folder)
94	2	10	single	5.0-5.0	32	2.5	0	0	N/A	N/A	0.2	median	3960	500	AdamW	0.001	0	64	97.16	77.02	221	95.71	benchmark for below test
95	2	10	single	5.0-5.0	32	2.5	0	0	0.05	5	0.2	median	3930	500	AdamW	0.001	0	64	95.83	76.59	227	96.69	introduction of removal of "silent" thresholds
96	2	10	single	5.0-5.0	32	2.5	0	0	0.2	5	0.2	median	3930	500	AdamW	0.001	0	64	97.58	77.48	219	96.31	
97	2	10	single	5.0-5.0	32	2.5	0	0	0.2	N/A	0.2	median	3880	500	AdamW	0.001	0	64	98.26	80.03	215	96.65	
98	2	10	single	5.0-5.0	32	2.5	0	0	0.2	N/A	0.2	median	3880	500	AdamW	0.001	0.5	64	96.59	88.02	221	98.71	
99	2	50	single	5.0-5.0	32	2.5	0	0	0.2	5	0.2	median	22300	500 (31)	AdamW	0.001	0.5	64	69.37	59.46	231	81.86	
100	2	50	single	5.0-5.0	32	2	0	0	0.2	5	0.2	median	26800	500 (42)	AdamW	0.001	0	64	93.69	57.2	374	77.50	increased experimental threshold to 10
101	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500 (59)	AdamW	0.001	0	128	98.14	63.66	3357	79.76	
102	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500 (66)	AdamW	0.001	0	128	97.77	63	59	80.39	
103	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500 (39)	AdamW	0.001	0	512	99.03	68.13	26	83.36	
104	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0	512	99.44	68.15	330	82.31	
105	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	512	91.14	75.65	335	90	
106	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	1024	91.89	76.77	328	90.32	
107	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	2048	93.26	76.32	327	90.09	
108	2	100	single	5.0-5.0	32	2	20	0	0.2	N/A	0.2	median	33900	500	AdamW	0.001	0.5	1024	93.19	74.38	420	87.07	

109	2	200	single	5.0-5.0	32	2	20	0	0.2	N/A	0.2	median	66400	500	AdamW	0.001	0.5	1024	88.17	70.29	804	83.48	changed code to be more memory efficient. should NOT cause memory leaks*
110	2	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	100	AdamW	0.001	0.5	1024	86.3	72.43	2640	88.43	benchmark for below test
111	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	50	AdamW	0.001	0.5	1024	86.81	73.06	4505	88.43	introduction of new model. Introduces ReLU after each linear layer
112	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	100	AdamW	0.001	0.5	1024	83.79	67.54	715	77.02	benchmark for below test
113	3	50	single	5.0-5.0	16	2	0	0	0.2	N/A	0.2	median	27050	100	AdamW	0.001	0.5	1024	79.88	66.64	86	76.04	decreasing of sample rate, 32->16khz
114	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	1024	83.58	72.78	351	77.95	above 2 tests but on SCIAMA
115	3	50	single	5.0-5.0	16	2	0	0	0.2	N/A	0.2	median	27050	500	AdamW	0.001	0.5	1024	83.82	71.15	188	79.39	
116	3	200	single	5.0-5.0	16	2	50	0	0.2	N/A	0.2	median	11120 0	500	AdamW	0.001	0.5	1024	69.67	56.53	746	62.18	
118	3	400	single	5.0-5.0	16	2	50	0	0.2	N/A	0.2	median	22800 0	500	AdamW	0.001	0.5	1024	58.46	46.62	1530	51.72	
119	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	1024	82.43	71.62	342	77.37	benchmark for below test
120	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.5	1024	83.55	71.96	340	78.15	scaled mel spectrogram values to be between 0-80 instead of all over the place
121	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.4	1024	85.33	71.81	340	80.99	
122	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.3	1024	89.9	75.76	337	84.59	
123	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.2	1024	80.96	67.71	338	73.97	
124	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0.1	1024	86.84	70.82	338	79.24	
125	3	50	single	5.0-5.0	32	2	0	0	0.2	N/A	0.2	median	26800	500	AdamW	0.001	0	1024	80.03	64.74	326	72.41	
126	3	50	single	5.0-5.0	32	2	0	0	1	5	0.2	median	25000	500	AdamW	0.001	0.5	1024	73.82	62.36	307	67.08	mean and variance threshold reintroduced, new settings in place

127	3	50	single	5.0-5.0	32	2	0	0	1	5	0.2	median	25000	500	AdamW	0.001	0.5	2048	85.7	71.26	299	78.80	
128	3	50	single	5.0-5.0	32	2	0	0	0.2	5	0.2	median	26800	500	AdamW	0.001	0.5	2048	85.82	72.93	324	80.04	
129	3	50	single	5.0-5.0	32	2	20	0	0.2	5	0.2	median	17950	500	AdamW	0.001	0.5	2048	85.23	75.49	222	80.59	
130	3	50	single	5.0-5.0	32	2	0	0	0.2	10	0.2	median	26800	500	AdamW	0.001	0.5	2048	88.05	74.65	327	82.05	
131	3	50	single	5.0-5.0	32	2	20	0	0.2	10	0.2	median	17950	500	AdamW	0.001	0.5	2048	85.31	75.74	216	81.34	
132	3	50	single	5.0-5.0	32	2	0	0	0.2	15	0.2	median	26800	500	AdamW	0.001	0.5	2048	85.97	72.91	325	80.06	
133	3	50	single	5.0-5.0	32	2	20	0	0.2	15	0.2	median	17950	500	AdamW	0.001	0.5	2048	85.33	75.04	219	81.03	
134	3	50	single	5.0-5.0	32	2	0	0	0.2	20	0.2	median	26800	500	AdamW	0.001	0.5	2048	85.94	73.02	323	79.68	
135	3	50	single	5.0-5.0	32	2	20	0	0.2	20	0.2	median	17950	500	AdamW	0.001	0.5	2048	85.12	75.4	219	80.59	
136	3	50	single	5.0-5.0	32	2	20	0	0.2	5	0.2	median	18513	500	AdamW	0.001	0.5	2048	91.94	78.07	255	90.01	instead of removing silence, make it its own class and train the model on it.
137	3	50	single	5.0-5.0	32	2	0	0	0.2	5	0.2	median	27693	500	AdamW	0.001	0.5	2048	85.91	71.31	369	83.37	
138	3	50	single	5.0-5.0	32	2	20	0	0.2	5	0.2	median	18513	500	AdamW	0.001	0.5	2048	86.41	72.94	222	79.69	relu6 instead of relu activation where seen
139	3	50	single	5.0-5.0	32	2	20	0	0.2	5	0.2	median	17950	500	AdamW	0.001	0.5	2048	86.35	74.82	216	79.16	above test, but without creating SILENT class

Appendix D: Sound Byte Predictions per Species

Index	Species	Total Top-5 Predictions	Total Sound Clips	Top-5 Accuracy (%)
10	Short-eared Owl	0	67	0.00
18	Indian Scops-Owl	0	72	0.00
23	Verdin	59	76	77.63
46	Phainopepla	52	65	80.00
4	Indigo Bunting	60	73	82.19
48	Red Crossbill	61	74	82.43
37	Loggerhead Shrike	66	80	82.50
32	Yellow-headed Blackbird	59	71	83.10
13	Yellow-headed Caracara	58	69	84.06
36	Bank Swallow / Sand Martin	58	67	86.57
25	Brubru	52	60	86.67
30	Least Tern	59	68	86.76
44	Olivaceous Woodcreeper	64	73	87.67
43	Ochre-bellied Flycatcher	67	76	88.16
49	Grey Catbird	65	73	89.04
38	Common Bulbul	57	64	89.06
19	Anna's Hummingbird	69	77	89.61
40	White-bearded Manakin	80	89	89.89
3	Barn Swallow	65	72	90.28
41	Common Pauraque	65	72	90.28
39	Brown Creeper	75	83	90.36
28	Whiskered Wren	68	75	90.67
33	Royal Tern	64	70	91.43
6	Mitred Parakeet	78	85	91.76
5	Common Loon	68	74	91.89

16	Spotted Towhee	59	64	92.19
45	Eurasian Collared-Dove	61	66	92.42
17	Blue-winged Warbler	61	66	92.42
9	Brown Jay	61	66	92.42
42	Piratic Flycatcher	65	70	92.86
29	Yellow-bellied Sapsucker	81	87	93.10
26	Black Francolin	69	74	93.24
11	Rosy-faced Lovebird	71	76	93.42
8	American Pipit	61	65	93.85
35	Scaly-breasted Wren	61	65	93.85
34	Long-tailed Duck	77	82	93.90
22	Little Egret	77	82	93.90
20	Black-winged Kite	54	57	94.74
21	Black-billed Magpie	82	86	95.35
0	Downy Woodpecker	63	66	95.45
1	Red-rumped Swallow	85	89	95.51
14	Curve-billed Thrasher	73	76	96.05
50	No Bird - Silence	65	67	97.01
47	Malabar Whistling-Thrush	67	69	97.10
24	Common Yellowthroat	69	71	97.18
31	Pied Bushchat	69	71	97.18
2	Common Chlorospingus	80	82	97.56
15	American Tree Sparrow	73	74	98.65
27	Common Buzzard	73	74	98.65
12	Zebra Dove	66	66	100.00
7	Blue-headed Parrot	67	67	100.00

Appendix E: Species File Accuracy

Index	Species	Total Files	Accuracy (%)	Top-5 Accuracy (%)
0	Downy Woodpecker	27	0.00	85.19
1	Red-rumped Swallow	66	0.00	83.33
40	White-bearded Manakin	33	0.00	75.76
30	Least Tern	44	47.73	56.82
2	Common Chlorospingus	26	0.00	50.00
21	Black-billed Magpie	13	0.00	38.46
5	Common Loon	10	0.00	20.00
20	Black-winged Kite	67	0.00	19.40
37	Loggerhead Shrike	38	0.00	13.16
36	Bank Swallow / Sand Martin	38	7.89	7.89
41	Common Pauraque	105	0.95	7.62
6	Mitred Parakeet	14	0.00	7.14
24	Common Yellowthroat	78	1.28	6.41
13	Yellow-headed Caracara	36	0.00	5.56
47	Malabar Whistling-Thrush	18	0.00	5.56
33	Royal Tern	21	0.00	4.76
19	Anna's Hummingbird	24	0.00	4.17
28	Whiskered Wren	41	0.00	2.44
3	Barn Swallow	232	0.00	2.16
16	Spotted Towhee	128	0.00	1.56
38	Common Bulbul	69	0.00	1.45
48	Red Crossbill	281	0.00	0.36
17	Blue-winged Warbler	11	0.00	0.00
15	American Tree Sparrow	14	0.00	0.00

8	American Pipit	24	0.00	0.00
9	Brown Jay	23	0.00	0.00
7	Blue-headed Parrot	31	0.00	0.00
4	Indigo Bunting	40	0.00	0.00
14	Curve-billed Thrasher	32	0.00	0.00
12	Zebra Dove	12	0.00	0.00
10	Short-eared Owl	22	0.00	0.00
11	Rosy-faced Lovebird	13	0.00	0.00
25	Brubru	9	0.00	0.00
18	Indian Scops-Owl	11	0.00	0.00
32	Yellow-headed Blackbird	23	0.00	0.00
31	Pied Bushchat	21	0.00	0.00
27	Common Buzzard	130	0.00	0.00
29	Yellow-bellied Sapsucker	14	0.00	0.00
26	Black Francolin	12	0.00	0.00
22	Little Egret	116	0.00	0.00
23	Verdin	31	0.00	0.00
35	Scaly-breasted Wren	31	0.00	0.00
39	Brown Creeper	32	0.00	0.00
34	Long-tailed Duck	20	0.00	0.00
42	Piratic Flycatcher	35	0.00	0.00
43	Ochre-bellied Flycatcher	10	0.00	0.00
45	Eurasian Collared-Dove	104	0.00	0.00
44	Olivaceous Woodcreeper	42	0.00	0.00
46	Phainopepla	12	0.00	0.00
49	Grey Catbird	56	0.00	0.00